

Project Documentation

Scope · architecture · data dictionary · ETL · model · DAX · report · DQ · build log

Fictional company · synthetic seeded data · generated 2026-09-10

00 — Scope & Scaffold (Phase 0)

Project: SwiftBite Delivery — Marketplace Liquidity & the Zone-Hour Incentive Experiment **Fictional company:** SwiftBite Delivery — a Brazilian food-delivery marketplace, single metro. **Two-sided marketplace:** couriers = supply · orders = demand. **Date locked:** 2026-09-10

PT — resumo. Projeto de portfólio nº 4. Marketplace fictício de delivery de comida ("SwiftBite"), operando em **uma** cidade brasileira dividida em **12 zonas**, moeda **BRL apenas**. Em horários de pico e em zonas específicas, a oferta de entregadores não acompanha a demanda → ETA alto, cancelamento e entregadores ociosos na zona errada (liquidez ruim). Operações testou um **incentivo dinâmico por zona-horário** (bônus temporário pago ao entregador para entrar numa zona/janela) como **experimento randomizado por zona-dia**. A pergunta: o incentivo melhora a liquidez o suficiente para pagar o próprio custo, funciona só nas zonas carentes, e há canibalização das zonas vizinhas?

1. Premise

SwiftBite runs a food-delivery marketplace in one large Brazilian metro. It does not employ couriers; they log into the courier app and choose when and where to work. On a normal weekday afternoon the marketplace clears fine. But at **dinner peak on Friday and Saturday**, and in a handful of **structurally supply-short zones**, demand outruns the couriers available: order-to-courier assignment slows, **ETA climbs**, a wave of orders is **cancelled** (by the customer while waiting, or by SwiftBite when no courier can be assigned), and — the marketplace-liquidity signature — **couriers sit idle in other zones** at the same moment, logged in but without a run.

The Operations team piloted a **dynamic zone-hour incentive**: a temporary per-delivery bonus, announced in-app, paid to couriers who complete deliveries in a **named zone during a named peak block**. It was run as a **randomized experiment with the zone-day as the unit** (not the user, not the order): on a treated zone-day the incentive is armed for that zone's peak blocks; on a control zone-day it is not.

This project reads out that experiment — **did the incentive buy enough liquidity to justify the bonus spend, is the effect concentrated in the supply-short zones, and does it cannibalise supply from neighbouring zones** — plus a Power BI report on marketplace health and courier economics.

2. Locked scope decisions

Dimension	Decision
Deliverable language	English (all docs, code, reports). Every deliverable also carries a short PT explanation of each step. Working chat in Portuguese.
Priority	Phase 10 (data analysis + the zone-hour incentive experiment) is the primary deliverable. Power BI track is secondary, scoped to 4 pages.
Geography / currency	One Brazilian metro (modelled on Belo Horizonte), 12 delivery zones, BRL only . No second city, no multi-currency — that ground is already covered by VoltEdge.
Data window	20 months: 2025-01-01 → 2026-08-31. Baseline period = months 1–15; experiment sits in months 16–18; post-window observation to month 20.
Experiment window	10 weeks: 2026-04-06 → 2026-06-14 (window-months 16–18) — a shoulder stretch, no major holiday inside it. Post-window Jun 15 – Aug 31 reserved for persistence / courier-supply after-effects.
Unit of randomization	Zone-day (a zone × a calendar date). Secondary cut: zone × hour-block . Not user, order or courier.
ETL / data-quality depth	Balanced (matches the other three): ~6 simulated source systems, ~8 data-quality issue classes (\$5).
Marketplace scale	~ 12 zones ; ~ 3,500 couriers ever active, ~ 900 active in a typical week; ~ 14–16k orders/day metro-wide (~ 7.5M orders over the window).
Experiment scale	Peak-eligible zone-days only (dinner blocks, all days): 12 zones × ~70 window-days ≈ 840 zone-days , 50/50 treated/control, stratified by zone and by baseline supply-stress tier .
Power BI track	Semantic model as code (TMDL) + PBIR + <code>theme.json</code> + build script; 4 pages , one business question per page.
Visual identity	New palette, designed in Phase 5 — must not reuse AeroVanti (navy/amber), VoltEdge (navy) or Voxa+ (violet).

3. Phase-1 report pages (4 pages, one question each)

#	Page	The question it answers
1	Executive Overview	Is the marketplace healthy overall, and did the zone-hour incentive work well enough to keep?
2	Marketplace Health	<i>Where and when</i> does liquidity break — fulfillment rate, ETA, idle couriers vs unmet demand, by zone × hour?
3	Pricing & Incentive Experiment	Is the incentive effect real, significant and safe on the guardrails — including neighbour-zone cannibalisation — or is one zone carrying it?
4	Courier Economics	What do couriers earn, how utilised are they, and how elastic is courier supply to the incentive (R\$ of bonus per incremental delivered order)?

4. Experiment design (locked at Phase 0, detailed in Phase 1)

Element	Decision
Unit of randomization	Zone-day . On a treated zone-day, the dynamic incentive is armed for that zone's peak blocks (18:00–22:00). Secondary analysis at zone × hour-block .
Eligible units	Peak-eligible zone-days : every zone, every date in the 10-week window, restricted to the dinner peak (the only time the incentive is offered). ~840 zone-days.
Assignment	Stratified randomization : within each zone and each baseline supply-stress tier (short / balanced / long, fixed from months 1–15), zone-days are split 50/50 treated/control. No day is switched mid-day; no re-randomization.
Treatment	A per-delivery bonus (a flat R\$ amount, set in <code>etl/config.py</code>) for deliveries completed in the treated zone during the peak block , plus an in-app "surge zone" push to nearby couriers.
Control	No bonus, no push — normal marketplace.
Primary metric	Fulfillment rate = orders delivered ÷ orders placed, at the zone-day (peak-block) grain.
Guardrail metrics	(1) Incentive cost per incremental delivered order (BRL); (2) ETA p50 and p90; (3) cancellation rate (customer-, courier- and no-courier-initiated); (4) courier earnings per active hour ; (5) neighbour-zone spillover — fulfillment rate and idle-courier ratio in <i>adjacent, control</i> zones on treated zone-days (the cannibalisation check).
Analyses required	Balance / sample-ratio check at zone-day grain; stated hypothesis + MDE / power computed for the zone-day unit (small N, intra-zone correlation → cluster-robust SEs and a randomization-inference check); primary effect with a 95% CI; heterogeneity by baseline supply-stress tier and by zone , with a formal arm × tier interaction test ; spatial-spillover / cannibalisation estimate with a stated sign and bound; persistence in the post-window weeks.

Underlying true effect: decided during Phase 3 (ETL) and baked into the generated data with plausible noise, a modest **neighbour-zone cannibalisation** term, and heterogeneity by supply-stress tier. The true values live only in `etl/config.py`; Phase 10 must detect and quantify them as if blind.

5. Data-quality issues to inject (balanced set)

1. **Multiple date/time formats** across source exports — DD/MM/YYYY, ISO, and Unix-epoch milliseconds in the GPS ping log.
2. **Timezone drift** — one source (payments) logs in UTC while everything else is `America/Sao_Paulo`; no offset column.
3. **Money as strings** — `"R$ 12,90"`, `"12.90"`, `"1.234,56"` mixed in payout and basket-value fields.
4. **GPS noise & out-of-area pings** — lat/long jitter, a tail of pings hundreds of km away (device cold-start), and (0, 0) nulls.
5. **Duplicate rows** — the consumer app retries on flaky mobile connections, so a slice of `orders` and `order_status_events` rows are exact or near-exact duplicates.
6. **Missing master rows** — some `courier_id`s in the trip log have no row in the courier dimension export (referential gap).
7. **Zone boundary redraw** — SwiftBite re-cut two zone polygons at window-month 9; a **current** zone-assignment table disagrees with the **as-booked** zone on older orders.
8. **Mojibake / mixed encoding** in customer names, address strings and restaurant names.

6. Simulated source systems

System	Feeds	Grain
Consumer order app	<code>orders</code> , <code>order_status_events</code> , basket value, address → zone	order / status event
Courier app	<code>courier_sessions</code> (login/logout), <code>shift_zone_blocks</code> , offer → accept/reject/timeout	session / offer
GPS / location service	<code>location_pings</code> (courier position over time), used to derive idle vs en-route minutes	ping (~30–60 s)
Dispatch / assignment engine	offer→assignment→pickup→dropoff timestamps, assignment latency, "no courier found" events	assignment
Payments & payouts	order payment, courier payout, incentive bonus payout	transaction
Incentive / campaign system	<code>incentive_campaigns</code> , per zone-day arm + bonus params + spend	zone-day
Zone / geo reference	zone polygons, adjacency graph, area, baseline supply-stress tier	zone (+ history)

7. Out of scope (explicit)

- A second city, region expansion, multi-currency / FX (covered by VoltEdge).
- Restaurant-side economics (menu pricing, commission take-rate optimisation), except as an input to demand.
- Consumer-side pricing / promo experiments (delivery fee, discount coupons) — the intervention studied here is **supply-side**.
- Real-time dispatch-algorithm design; routing optimisation.
- A courier-churn *prediction* model — Phase 10 quantifies supply elasticity and the experiment effect; it does not ship a scorecard.
- Fraud (GPS spoofing, incentive gaming) beyond a noted data-quality tail.
- Real personal data — every courier, customer, order, ping and payout is synthetic and seed-generated.
- The **true incentive effect size and cannibalisation term** — decided in Phase 3, not here.

8. Resolved-assumptions log

#	Assumption	Resolution
A1	City & geography	One metro (modelled on Belo Horizonte), 12 zones , adjacency graph fixed in the geo reference (with one mid-window redraw as a DQ issue).
A2	Window & anchor	20 months, 2025-01-01 → 2026-08-31; data-gen date 2026-09-10.
A3	Baseline supply-stress tier	Each zone tagged short / balanced / long on supply, fixed from months 1–15 behaviour; used for stratification and heterogeneity.
A4	"Active courier"	≥ 1 completed delivery in the trailing 7 days (weekly) / trailing 28 days (monthly cut).
A5	Liquidity metric	Available courier-hours ÷ demanded delivery-hours, by zone × hour; < 1 = supply-short.
A6	Fulfillment rate	Orders delivered ÷ orders placed, same zone-hour; the experiment's primary metric.
A7	ETA	Order-placed → delivered wall-clock minutes; report p50 and p90 (p90 is the customer-pain metric).
A8	Cancellation taxonomy	customer-initiated · courier-initiated (after accept) · no-courier-found (system).
A9	Experiment unit / split	Zone-day , 50/50, stratified by zone × baseline supply-stress tier; peak block 18:00–22:00.
A10	Experiment window	10 weeks, 2026-04-06 → 2026-06-14 (window-months 16–18); post-window to 2026-08-31.
A11	Primary metric & decision rule	Fulfillment rate; ship if lift is positive, significant at $\alpha = 0.05$ (cluster-robust), ≥ MDE, and incentive cost per incremental delivered order is below the stated threshold.
A12	Guardrails	Incentive cost / incremental order; ETA p50 & p90; cancellation rate; courier earnings / active hour; neighbour-zone spillover .

#	Assumption	Resolution
A13	MDE / power	Computed for the zone-day unit ($N \approx 420/\text{arm}$), with intra-zone ICC; randomization-inference cross-check. Planning values finalized in Phase 1.
A14	Cannibalisation	A real, modest negative spillover to adjacent control zones is baked into the data in Phase 3; Phase 10 must estimate its sign and size.
A15	Currency	BRL only; no FX.
A16	True effect	Not decided here; set in ETL (Phase 3), detected blind in Phase 10.

9. Folder scaffold (created)

```
SwiftBite-Delivery-Analytics/  
docs/          00 scope · 01 business context & KPIs · 02 architecture · 03 data dict ·  
               04 ETL · 05 model · 06 DAX · 07 visual identity · 08 data quality ·  
               09 report structure · walkthrough.md · docs bundle  
etl/           config (one SEED) · gen/ · 01 generate raw · 02 clean/stage ·  
               03 build curated · dq_checks · doc + DAX generators · run_pipeline  
data/  
  raw/         source-faithful messy exports          (git-ignored, regenerated)  
  staging/      typed stg_* tables                    (git-ignored, regenerated)  
  curated/      Kimball star, CSV + Parquet            (git-ignored, regenerated)  
  quality/      dq_report.md  
powerbi/       semantic model as code (TMDL) · PBIR report · theme · build script  
assets/        brand.py · logo / page-background generators  
data_analysis/ sql/ python/ findings/ parity/ deliverables/ outputs/ ← PRIMARY  
portfolio/     5 gallery images only  
README.md
```

End of Phase 0 deliverable. Next gate: Phase 1 (docs/01_business_context_kpis.md) — drafted alongside this; review both before Phase 2.

SwiftBite Delivery — Business Context & KPI Framework

Document status: Phase 1 deliverable · authored at project kickoff, before any analysis. **Data generation date:** 2026-09-10 · **Marketplace-history window:** 2025-01-01 → 2026-08-31 (20 full months). **Embedded experiment:** the zone-hour incentive test, 2026-04-06 → 2026-06-14 (10 weeks, window-months 16–18). **Audience:** SwiftBite's Operations, Growth and Finance leads.

This document frames the problem and defines the metrics. It deliberately does **not** state whether the zone-hour incentive worked, or by how much — that is the job of the read-out (Phase 10). Everything here was knowable *before* the analysis. The true underlying treatment effect — including the neighbour-zone cannibalisation term — is decided during ETL (Phase 3), baked into the synthetic data with noise and heterogeneity, and kept only in `etl/config.py`; Phase 10 must detect and quantify it as if blind.

PT — o que este documento é. Enquadra o problema de negócio e define as métricas de um marketplace de delivery. **Não** diz se o incentivo por zona-horário funcionou — isso é a Fase 10. O efeito verdadeiro (e a canibalização entre zonas vizinhas) é plantado nos dados sintéticos na Fase 3 e fica só em `etl/config.py`; a análise tem que recuperá-lo "às cegas".

1. The situation

PT. SwiftBite é um marketplace de delivery de comida numa única cidade grande do Brasil. Não emprega entregadores — eles entram no app e escolhem quando e onde rodar. Em **pico de jantar de sexta e sábado**, e em algumas **zonas cronicamente com pouca oferta**, a demanda passa da oferta: a fila de atribuição atrasa, o **ETA sobe**, uma onda de pedidos é **cancelada**, e — a assinatura de liquidez ruim — há **entregadores ociosos noutras zonas** ao mesmo tempo. Operações testou um **bônus dinâmico por zona-horário** como experimento randomizado por **zona-dia**.

SwiftBite Delivery operates a food-delivery marketplace in one large Brazilian metro (modelled on Belo Horizonte). It is a **two-sided marketplace**:

- **Demand** — consumers place orders in the SwiftBite app; each order belongs to a **delivery zone** (from the delivery address) and a **time block**.
- **Supply** — independent couriers log into the courier app and choose their hours and rough working area. SwiftBite does not roster them; it can only *influence* where supply goes — with information, with assignment logic, and with money.

Most of the week, the metro clears fine: a courier is assigned in a minute or two, ETA is 30–40 minutes, almost every order is delivered. The problem is **concentrated in time and space**:

1. **Peak dinner on Friday and Saturday** (≈ 18:00–22:00) — order volume roughly triples over a weekday lunch, but courier supply rises far less.
2. **A handful of structurally supply-short zones** — outer-ring residential zones with long trips, poor return-load prospects and few couriers who live nearby.

When those overlap, three things happen together and define **poor marketplace liquidity**:

Symptom	Metric it shows up in
Assignment slows / fails	Assignment latency ↑, "no courier found" cancellations ↑
Customers wait, then bail	ETA p90 ↑, customer-initiated cancellation rate ↑
Supply is in the wrong place	Idle-courier ratio ↑ in <i>other</i> zones at the same minute
Orders never delivered	Fulfillment rate ↓ (the primary experiment metric)

The Operations team's hypothesis: a **temporary, targeted per-delivery bonus** — announced in-app for a **named zone during the peak block** — will pull enough nearby couriers into that zone to lift fulfillment, at a bonus cost lower than the contribution margin of the orders it saves. Growth's worry: the couriers who move are **taken from adjacent zones**, so the local win is partly a transfer, not new supply. Finance's worry: the bonus is paid on **every** delivered order in the zone-block, including the ones that would have happened anyway. The experiment is designed to answer all three.

2. Company profile

Name	SwiftBite Delivery (legal entity <i>SwiftBite Tecnologia e Logística Ltda.</i>)
Headquarters	Belo Horizonte (MG), Brazil
Business	On-demand food-delivery marketplace — consumers ↔ restaurants ↔ independent couriers
Footprint in scope	One metro, 12 delivery zones. ~6,000 active restaurants.
Age	Founded 2022; this project's 20-month window is 2025-01 → 2026-08 (a mature, steady operation, not a launch ramp).
Currency	BRL only. Single-country, single-city; no FX in scope.
Reporting calendar	Calendar months for finance; operations reporting is by day and by hour-block ; the experiment is analysed by zone-day.

Scale at window end	~ 3,500 couriers active over the window (~ 900 in a typical week); ~ 15,000 orders/day metro-wide (~ 7.5M orders across the window); average basket ~ R\$ 62 ; average delivery fee ~ R\$ 8.50 ; SwiftBite take rate ~ 22% of basket + fee share.
---------------------	--

Marketplace positioning

SwiftBite is the **#2 player** in this metro behind a national incumbent. It competes on **ETA reliability** and **courier earnings** (couriers multi-app; the platform with better peak earnings gets their hours). Restaurant commission is not a lever in this project — it is treated as a fixed input to unit economics.

3. The two sides of the marketplace

PT. Os dois lados nunca devem ser misturados na análise: **oferta** = horas de entregador disponíveis; **demand** = pedidos (ou horas-entrega demandadas). Liquidez é a razão entre os dois num dado par zona-hora.

3.1 Supply — couriers

Concept	Definition
Courier session	A continuous logged-in period in the courier app (login → logout).
Active minutes	Session minutes assigned to an offer or en route (offer→accept→pickup→dropoff).
Idle minutes	Session minutes logged in but not on a run and not in cooldown. High idle = supply present but unproductive (often in the wrong zone).
Available courier-hours (a zone-hour)	Σ session-hours whose position is in the zone during the hour, net of cooldown. The supply quantity.
Active courier	≥ 1 completed delivery in the trailing 7 days (weekly cut) / 28 days (monthly cut).
Courier home zone	Modal drop-off zone over the trailing 28 days — used only for descriptive supply geography, never for assignment.
Multi-apping	Assumed but not observed directly; shows up as couriers going idle-then-logout during a SwiftBite lull. A stated limitation.

3.2 Demand — orders

Concept	Definition
Order	A placed consumer order with a delivery address → delivery zone and a time block .
Demanded delivery-hours (a zone-hour)	Orders in the zone-hour $\times$ the period's average service time (pickup + drive + drop). The demand quantity, in the same unit as supply.
Order outcome	delivered · cancelled_customer · cancelled_courier · cancelled_no_courier · cancelled_restaurant .
ETA	Order-placed → delivered wall-clock minutes. Reported p50 (typical) and p90 (the customer-pain tail).
Assignment latency	Order-ready → courier-assigned seconds.

3.3 Liquidity — where the two meet

Metric	Definition	Read
Liquidity ratio	available courier-hours $\div$ demanded delivery-hours, per zone $\times$ hour	< 1 → supply-short; the marketplace cannot clear without ETA blowing out or orders dropping
Fulfillment rate	orders delivered $\div$ orders placed, same zone-hour	the headline outcome — and the experiment's primary metric
Idle-courier ratio	idle courier-hours $\div$ available courier-hours, per zone $\times$ hour	high and low fulfillment in a <i>neighbouring</i> zone at the same time = a spatial mismatch the incentive is meant to fix
Unmet demand	placed – delivered, per zone-hour	the volume the incentive is trying to recover

4. Zones & geography

PT. 12 zonas cobrindo a cidade; cada zona tem vizinhas (grafo de adjacência) e um **tier de estresse de oferta** (curta / equilibrada / longa) fixado pelo comportamento dos meses 1–15. O tier é usado para estratificar o experimento e para a análise de heterogeneidade. A canibalização é justamente supor que o incentivo numa zona rouba oferta das **zonas adjacentes**.

Property	Detail
Zone count	12, tiling the metro; each is a group of neighbourhoods with one dispatch centroid.
Adjacency graph	Fixed in the geo reference; each zone has 2–5 neighbours. Cannibalisation is defined against adjacent zones .

Property	Detail
Baseline supply-stress tier	Each zone tagged short / balanced / long from months 1–15 liquidity (4 short · 5 balanced · 3 long, indicative). Fixed pre-experiment; used for stratification and heterogeneity.
Zone boundary redraw	Two zone polygons were re-cut at window-month 9 (Sep 2025) — a deliberate data-quality issue: the current zone map disagrees with the as-booked zone on older orders (§ data-quality doc).
Trip geography	"short" zones have longer average trips and lower return-load probability — the structural reason supply avoids them.

5. The zone-hour incentive experiment

5.1 Design

PT. Unidade de randomização = **zona-dia** (uma zona num dia). Em zona-dia tratado, o bônus por entrega fica ligado no bloco de pico (18:00–22:00) daquela zona; em controle, não. Split 50/50, **estratificado por zona e por tier de estresse de oferta**. Nada muda no meio do dia; sem re-randomização.

Element	Decision
Unit of randomization	Zone-day — one zone on one calendar date. Secondary analysis at zone × hour-block .
Eligible units	Peak-eligible zone-days: every zone, every date in the 10-week window, peak block 18:00–22:00 (the only time the incentive is ever offered). ≈ 840 zone-days (12 × 70).
Assignment	Stratified randomization : within each zone and each baseline supply-stress tier , zone-days split 50/50 treated/control, balanced across day-of-week (so treated and control both get their share of Fri/Sat). Assignment fixed at 00:00 local; no mid-day switch; no re-randomization.
Treatment	A flat per-delivery bonus (R\$ amount set in <code>etl/config.py</code>) for orders completed in the treated zone during the peak block , plus an in-app "surge zone" push to couriers within ~2 km.
Control	No bonus, no push — the normal marketplace.
Blinding	Couriers see the bonus (it must be visible to work); they do not see the experiment. Customers see nothing.
Duration	10 weeks, 2026-04-06 → 2026-06-14. Post-window observation to 2026-08-31 for persistence.

5.2 Hypotheses

- **Business hypothesis.** Arming a targeted peak-block bonus in a zone raises **fulfillment rate** in that zone-block (more couriers present → fewer no-courier cancellations, lower ETA, fewer customer bail-outs), at a **bonus cost per incremental delivered order below the contribution margin of a saved order**, and the effect is **largest in supply-short zones**. A secondary hypothesis is that some of the gained supply is **drawn from adjacent zones** (cannibalisation), so the metro-level net effect is smaller than the sum of local effects.
- **Statistical hypothesis (primary).** H_0 : fulfillment rate is equal on treated and control zone-days over the window. H_1 : treated > control. Tested one-sided at $\alpha = 0.05$ with **cluster-robust (zone) standard errors**; a two-sided 95% CI on the difference is always reported, alongside a **randomization-inference** p-value as a small-N cross-check.

5.3 Metrics

Role	Metric	Definition	Decision rule
Primary	Fulfillment rate	Orders delivered ÷ orders placed, treated zone-days vs control zone-days, peak block.	Keep the incentive if lift is positive, significant ($\alpha = 0.05$, cluster-robust), ≥ the MDE, and guardrail G1 passes.
Guardrail G1	Incentive cost per incremental delivered order (BRL)	Total bonus spend ÷ (incremental delivered orders vs control).	Ship only if below the contribution margin of a delivered order (planning value ≈ R\$ 9–11). Reported with a CI.
Guardrail G2	ETA p50 / p90 (min)	Placed→delivered minutes, by arm.	Directional; flag if treated is worse (should improve or hold).
Guardrail G3	Cancellation rate	Cancelled ÷ placed, split by cause (customer / courier / no-courier).	Flag if total or no-courier rate is worse on treated.
Guardrail G4	Courier earnings per active hour (BRL)	Payout incl. bonus ÷ active hours, couriers working the zone-block.	Context + fairness; expected up on treated (that is the mechanism).
Guardrail G5	Neighbour-zone spillover (cannibalisation)	Δ fulfillment rate and Δ idle-courier ratio in adjacent control zones on treated zone-days vs non-treated-neighbour zone-days.	Estimate sign and size ; flag if the metro-level net lift is < X% of the summed local lift.

5.4 Power & minimum detectable effect (zone-day unit)

PT. A unidade é **zona-dia**, então o N efetivo é pequeno (~420 por braço) e há **correlação intra-zona** — o cálculo de poder tem que usar essa unidade, não o número de pedidos. Erros-padrão com cluster por zona + inferência por randomização.

Assumption	Value (planning)
Analysis unit	zone-day (peak block)
Zone-days per arm	≈ 420 (≈ 840 eligible, 50/50)
Control fulfillment rate at peak, supply-short zones (planning value)	≈ 0.86
Between-zone-day SD of fulfillment rate (peak)	≈ 0.07 (finalized from realized baseline in Phase 10)
Intra-zone correlation (ICC)	assumed ≈ 0.15 → design effect ≈ 1.9 for ~35 days/zone/arm
α (two-sided) / power	0.05 / 0.80
MDE on fulfillment rate , all zones pooled	≈ +1.3 pp
MDE within the supply-short tier (≈ 140 zone-days/arm)	≈ +2.2 pp — the tier the decision hinges on; adequately but not generously powered
MDE within a single zone	≈ +4–5 pp — single-zone effects are directional only
Secondary unit (zone × hour-block)	~4× the rows, used for the novelty/decay and time-of-peak cut, not for the headline test

The power calculation is re-run from realized pre-period variance and reproduced in code in Phase 10; the values above are the planning basis.

5.5 Threats to validity the read-out must address

Threat	How it is handled
Assignment imbalance / SRM	χ^2 on treated/control counts overall, per zone, per supply-stress tier and per day-of-week; investigate if $p < 0.01$.
Covariate imbalance	Standardized differences on pre-period fulfillment, ETA, order volume, liquidity ratio, weather flag, day-of-week; Love plot.
Spatial spillover / cannibalisation (SUTVA violation)	This is a <i>studied effect</i> , not just a nuisance: G5 estimates it directly by comparing adjacent-control zones on treated vs non-treated-neighbour days. The primary estimate is also reported excluding zone-days adjacent to a treated zone, as a bound.
Novelty / courier learning	Weekly treated-vs-control lift across the 10 weeks; test for a decaying trend (weeks 1–3 vs 8–10); report the settled lift.
Weather & events	Rain and city-event flags as covariates; control zone-days are concurrent so metro-wide shocks difference out, but zone-local rain does not — hence the covariate.
Demand endogeneity	The bonus could itself change customer behaviour (faster ETA → more orders). Fulfillment rate is a ratio, but order <i>volume</i> is also reported by arm so a demand response is visible, not hidden.
Small number of clusters	12 zones → cluster-robust SEs are anti-conservative; a randomization-inference p-value and a wild-cluster bootstrap are reported alongside.
Multiple comparisons	One primary metric fixed in advance; 5 guardrails + the tier/zone heterogeneity cut are secondary and CI-reported with a family-wise-error note.
Persistence	The decision is on the in-window settled lift; the post-window weeks are reported as a persistence signal, not a second test.

5.6 Heterogeneity analysis (required)

Effect on fulfillment rate estimated **within each baseline supply-stress tier** (short / balanced / long) and **within each zone**, with per-cell CIs and a formal **arm × tier interaction** test. The prior to test — not assume — is that the effect is **largest in supply-short zones** (most unmet demand to recover) and **near zero in supply-long zones** (already clearing; the bonus is pure cost there). The operational decision — roll the incentive out everywhere, or only to short zones — hinges on this cut and on G1 computed per tier.

6. The analytics window — why 20 months

Anchor	Value
Window length	20 complete months: month 1 = Jan 2025 , month 20 = Aug 2026
Baseline period	months 1–15 (Jan 2025 – Mar 2026) — defines each zone's supply-stress tier
Experiment	window-months 16–18 → 2026-04-06 → 2026-06-14 (10 weeks)
Post-window observation	2026-06-15 → 2026-08-31 (~11 weeks) — persistence
Year-over-year	months 1–12 vs a trailing-12 ending at month 20, for the report's trend context only

Why 20 months. Long enough for two full seasonal cycles of the parts that matter (Carnaval, June festas, school holidays, rainy season) and a stable 15-month baseline before the experiment; the operation is mature over the whole window, so there is **no launch-ramp comparability caveat** (unlike AeroVanti). Shorter than VoltEdge's 24 because there is no multi-year financial trend to establish — the value is in the experiment.

7. Seasonality & patterns embedded in the data

Pattern	Where it shows
Weekly — Fri/Sat dinner peak	Order volume ~3× a weekday lunch; the peak block the experiment targets
Daily — lunch (11:30–14:00) and dinner (18:00–22:00) double peak	Dinner is longer, more supply-short
Rainy season (Oct–Mar in the Southeast)	Demand ↑ (people don't go out), courier supply ↓ (fewer want to ride) → the worst liquidity of the year; the experiment's first 2 weeks catch the tail of it
Carnaval (Feb/Mar)	Multi-day demand collapse then rebound — outside the experiment window by design
June festas / school July holidays	Elevated weekend demand; overlaps the experiment's back half — handled by the concurrent control arm
Payday (5th & 20th)	Mild order-volume and basket-size lift
Big football matches / shows	Zone-local demand spikes; flagged as an event covariate
Courier supply drift	Slow metro-wide decline in couriers/week across 2026 as a competitor raises pay — a backdrop trend, differenced out by the concurrent control

8. Stakeholder personas → report pages

PT. Três personas, uma pergunta permanente cada — mais a pergunta executiva. As quatro viram as quatro páginas do relatório.

#	Persona	Standing question	Page
1	Head of Operations	Is the marketplace healthy this week, and did the zone-hour incentive earn its cost?	Executive Overview
2	Marketplace / Liquidity Manager	Exactly which zones and hours are supply-short, how bad is the ETA and cancellation damage, and where is idle supply sitting while that happens?	Marketplace Health
3	Growth / Courier Supply Lead (<i>and the experimentation partner</i>)	Is the incentive effect real, significant and safe on the guardrails — including cannibalisation of neighbour zones — or is one zone carrying it?	Pricing & Incentive Experiment
4	Finance / Unit Economics	What do couriers earn per hour, how utilised is supply, and how many reais of bonus buy one extra delivered order?	Courier Economics

9. KPI framework

Benchmark ranges are **directional**, drawn from public delivery-marketplace disclosures and industry commentary (gig-economy and last-mile studies), adapted to a single-metro #2 player. They set targets; they are not claims about named competitors.

9.1 Marketplace health & demand

KPI	Definition	Benchmark	Target
Orders placed / day	Distinct placed orders	—	on plan
Fulfillment rate %	Delivered ÷ placed	95%–99%	≥ 98% metro; ≥ 95% in every zone-hour
ETA p50 / p90 (min)	Placed → delivered	p50 28–38 · p90 45–70	p90 ≤ 55
Cancellation rate %	Cancelled ÷ placed	2%–6%	≤ 3%
— of which no-courier %	<code>cancelled_no_courier</code> ÷ placed	0.3%–2%	≤ 0.5%
Assignment latency p90 (s)	Ready → assigned	60–240	≤ 150
Repeat-order rate % (28d)	Customers with ≥ 2 orders in 28d	—	context

9.2 Supply — couriers

KPI	Definition	Benchmark	Target
Active couriers / week	≥ 1 delivery in trailing 7d	—	≥ plan
New couriers / month	First delivery in the month	—	≥ churn
Courier CAC (BRL)	Acquisition + onboarding-bonus spend ÷ new activated couriers	—	≤ plan
Courier 4-week retention %	Active in week 4 after first delivery	45%–70%	≥ 60%
Available courier-hours / zone-hour	Σ session-hours in the zone-hour (net cooldown)	—	≥ demanded hours
Idle-courier ratio %	Idle ÷ available courier-hours	15%–35%	≤ 25% (and not concentrated)
Deliveries per active hour	Completed deliveries ÷ active hours	1.6–2.6	≥ 2.0 at peak

9.3 Liquidity (zone × hour — the core of page 2)

KPI	Definition	Benchmark	Target
Liquidity ratio	available courier-hours ÷ demanded delivery-hours	≥ 1.0	≥ 1.1 at peak in every zone
Supply-short zone-hours %	Share of peak zone-hours with ratio < 1	—	≤ 10%
Unmet demand / peak	placed – delivered, peak blocks	—	minimise
Spatial-mismatch index	corr. of (idle ratio in zone A) with (unmet demand in adjacent zone B), same hour	—	monitor; the incentive should reduce it
ETA gap (short vs long zones)	ETA p90 short-tier – ETA p90 long-tier	—	narrowing

9.4 Courier economics & unit economics (page 4)

KPI	Definition	Benchmark	Target
Courier earnings per active hour (BRL)	Payout incl. tips & bonus ÷ active hours	R\$ 22–34	≥ R\$ 26 at peak
Utilisation %	Active ÷ logged-in minutes	55%–80%	≥ 70% at peak
Payout per delivery (BRL)	Total payout ÷ deliveries	—	context
Contribution margin per delivered order (BRL)	Take (commission + fee share) – payout – payment & support cost	R\$ 6–14	≥ R\$ 9
Incentive spend (BRL)	Total dynamic-bonus payout	—	≤ budget
Incentive cost per incremental delivered order (BRL)	Bonus spend ÷ incremental delivered orders	—	< contribution margin (≈ R\$ 9–11)
Incentive spend as % of courier payout	Bonus ÷ total payout	—	≤ 8%
Supply elasticity to incentive	% change in available courier-hours in a zone-block per R\$1 of bonus	—	quantify (Phase 10)

9.5 Experiment scorecard (Pricing & Incentive Experiment page)

KPI	Definition
Fulfillment rate by arm (+ lift, cluster-robust CI, p, RI p)	Primary metric, \$5.3
Weekly fulfillment lift across the 10 weeks	Novelty / decay diagnostic, \$5.5
Balance check (SRM χ^2 + SMD / Love plot)	Randomization check
Incentive cost per incremental delivered order (+ CI)	Guardrail G1 — the go/no-go economic test
ETA p50 / p90 by arm	Guardrail G2
Cancellation rate by arm and by cause	Guardrail G3
Courier earnings per active hour by arm	Guardrail G4
Neighbour-zone spillover: Δ fulfillment & Δ idle ratio in adjacent control zones	Guardrail G5 — cannibalisation
Effect by supply-stress tier and by zone (+ arm × tier interaction test)	Heterogeneity, \$5.6
Metro-level net lift vs summed local lift	Cannibalisation-adjusted bottom line

10. Targets and plan

- The plan is built on the **same basis as the actuals** (same fulfillment, ETA, cancellation and contribution-margin definitions).
- **Plan fulfillment rate:** ≥ 98% metro-wide, ≥ 95% in every peak zone-hour.
- **Plan p90 ETA:** ≤ 55 min at peak in every zone.
- **Plan supply-short peak zone-hours:** ≤ 10%.
- **Plan incentive efficiency:** cost per incremental delivered order **below** the contribution margin of a delivered order.
- "vs Target" measures exist per KPI; **ETA, cancellation rate, idle ratio, no-courier rate, courier CAC and incentive cost per incremental order are scored lower-is-better.**

11. Multi-currency handling

Out of scope by design. All monetary values are **BRL**. No *_local / *_usd split, no FX table, no reporting-currency slicer. Multi-currency was the VoltEdge project's job; repeating it here would only distract from the marketplace analysis.

12. Out of scope (explicit)

- A second city, expansion modelling, multi-currency / FX.
- Restaurant-side economics (commission optimisation, menu pricing, prep-time SLAs) beyond a fixed input to demand and unit economics.
- Consumer-side pricing experiments (delivery fee, coupons, subscription) — this project's intervention is **supply-side only**.
- Dispatch / routing-algorithm design and real-time optimisation.
- A courier-churn or demand-forecast **model**. Phase 10 quantifies supply elasticity and the experiment effect; it does not ship a predictive scorecard.
- Fraud (GPS spoofing, incentive gaming, collusion) beyond a noted data-quality tail.
- Weather/traffic modelling — rain and events enter only as covariate flags.
- Real personal data. Every courier, customer, restaurant, order, GPS ping and payout is synthetic and seed-generated.
- The **true incentive effect size and the cannibalisation term** — decided in Phase 3, detected blind in Phase 10.

13. Resolved-assumptions log

#	Decision	Resolution
A1	Window length & anchor	20 months, 2025-01-01 → 2026-08-31; data-gen date 2026-09-10; mature operation throughout (no launch ramp).
A2	Geography	One metro (Belo Horizonte), 12 zones, fixed adjacency graph, one mid-window boundary redraw as a DQ issue.
A3	Baseline supply-stress tier	short / balanced / long, fixed from months 1–15 liquidity; ~4 / 5 / 3 zones.
A4	Two marketplace sides	Supply = available courier-hours; demand = demanded delivery-hours; never conflated.
A5	Active courier	≥ 1 completed delivery in trailing 7 days (weekly) / 28 days (monthly).
A6	Liquidity ratio	available courier-hours ÷ demanded delivery-hours, per zone × hour.
A7	Primary metric	Fulfillment rate = delivered ÷ placed, zone-day peak block.
A8	ETA	Placed → delivered minutes; p50 and p90 both reported.
A9	Cancellation taxonomy	customer · courier · no-courier · restaurant.
A10	Experiment unit / split	Zone-day , 50/50, stratified by zone × supply-stress tier, balanced on day-of-week; peak block 18:00–22:00.
A11	Experiment window	10 weeks, 2026-04-06 → 2026-06-14 (window-months 16–18); post-window to 2026-08-31.
A12	Treatment	Flat per-delivery bonus (R\$, in <code>etl/config.py</code>) for peak-block deliveries in the treated zone + in-app surge-zone push.
A13	Primary decision rule	Positive, significant ($\alpha = 0.05$, cluster-robust), ≥ MDE, and guardrail G1 (cost per incremental order) below the delivered-order contribution margin.
A14	Guardrails	G1 incentive cost / incremental order · G2 ETA p50/p90 · G3 cancellation rate by cause · G4 courier earnings / active hour · G5 neighbour-zone spillover.
A15	MDE / power	Computed for the zone-day unit (~420/arm), ICC ≈ 0.15, design effect ≈ 1.9; pooled MDE ≈ +1.3 pp, supply-short-tier MDE ≈ +2.2 pp; RI + wild-cluster bootstrap cross-checks. Finalized in Phase 10 from realized variance.
A16	Cannibalisation	A real, modest negative spillover to adjacent control zones baked into the data in Phase 3; Phase 10 estimates its sign and size and reports a cannibalisation-adjusted metro net lift.
A17	Required diagnostics	SRM/balance, covariate Love plot, novelty decay, tier & zone heterogeneity + interaction test, spillover estimate, persistence.
A18	Currency	BRL only; no FX in scope.
A19	Seasonality	Rainy-season and June-festas overlap the window edges; Carnaval deliberately excluded; concurrent control arm handles metro-wide shocks.
A20	True treatment effect	Deliberately not decided here; set in ETL (Phase 3), detected blind in Phase 10.

End of Phase 1 deliverable.

SwiftBite Delivery — Data Architecture

Document status: Phase 2 deliverable. **Depends on:** `00_scope.md` , `01_business_context_kpis.md` . **Feeds:** Phase 3 (ETL build), Phase 4 (data dictionary / model docs), Phase 6 (semantic model), **Phase 10 (analysis — primary deliverable)**.

This is a build spec: every layer, every table, every data-quality issue Phase 3 must inject and then handle. Scope note from Phase 0: the messy-data layer is **balanced** — 6 simulated operational systems + a static geo reference, 8 primary data-quality issue classes. Effort is concentrated on a curated layer that serves the **zone-hour incentive read-out** cleanly, at the zone-day and zone-hour grains the experiment needs.

PT — o que é este documento. Especificação de construção do pipeline: as três camadas (raw → staging → curated), cada tabela, e cada problema de dados que a Fase 3 injeta de propósito e depois trata. O alvo é uma camada curated limpa nos grãos **zona-dia** e **zona-hora**, que são as unidades do experimento.

1. Principles — medallion, seeded, idempotent

Data only ever flows downward.

Layer	Folder	Contract
Raw	<code>data/raw/<source>/</code>	Byte-faithful simulated exports — messy, locale-specific, duplicated, encoded however the fake source system would encode it. Never edited after generation. Regenerated only when <code>SEED</code> / <code>config.py</code> changes.
Staging	<code>data/staging/</code>	Exactly one typed table per raw entity: <code>stg_<source>_<entity>.parquet</code> . Parsing, type coercion, encoding repair, de-duplication, timezone normalisation, GPS sanitisation, key & category normalisation. No cross-source joins, no business logic, no metric definitions. Every fix counted into a DQ fragment.
Curated	<code>data/curated/</code>	Conformed Kimball star: <code>dim_*</code> / <code>fact_*</code> as Parquet and a CSV mirror. Surrogate keys, address→zone and GPS→zone resolution (both as-booked and current zone maps), supply/demand put in the same unit (hours), the zone-hour liquidity spine, and the pre-built experiment outcome tables at zone-day grain. All money in BRL .
Quality	<code>data/quality/</code>	DQ fragments from staging + curated, assembled into <code>dq_report.md</code> ; hard-assertion results from <code>dq_checks.py</code> .

Seeded. A single `SEED` in `etl/config.py` drives every random draw. Same seed → byte-identical raw → identical curated. **The true incentive effect and the neighbour-zone cannibalisation term live only in `config.py`** (see §10, B1).

Idempotent. Each script wipes and rebuilds its own layer. `run_pipeline.py` runs `01` → `02` → `03` → `dq_checks` end to end; `dq_checks.py` exits non-zero on any violated assertion.

Layered. Staging reads only raw. Curated reads only staging. The semantic model (Phase 6) and the analysis (Phase 10) read only `data/curated/*.parquet` .

2. Tech stack & run instructions

Concern	Choice
Language	Python 3.11+
Core libs	<code>pandas</code> , <code>numpy</code> , <code>pyarrow</code> (Parquet), <code>python-dateutil</code> , <code>openpyxl</code> (write messy <code>.xlsx</code>), <code>Faker</code> (<code>pt_BR</code> — names, addresses, restaurant names), <code>ftfy</code> + <code>unicodedata</code> (encoding repair), <code>shapely</code> (zone polygons + point-in-polygon for address→zone and ping→zone), <code>duckdb</code> (Phase 10 SQL track), <code>scipy</code> / <code>statsmodels</code> (Phase 10 tests — cluster-robust SEs, interaction tests; not used in ETL)
Storage	Flat files. Raw = <code>.csv</code> / <code>.xlsx</code> / <code>.csv.gz</code> / <code>.parquet</code> / <code>.ndjson.gz</code> (GPS). Staging & curated = <code>.parquet</code> (+ curated <code>.csv</code> mirror). No database server.
Orchestration	Plain scripts, ordered by <code>run_pipeline.py</code> . No Airflow / dbt.
Determinism	One <code>SEED</code> ; every generator derives a named sub-seed from it.
GPS volume	Raw pings are the biggest object by far (~30–60 s cadence × ~900 weekly couriers). <code>01</code> writes them partitioned by month as gzipped NDJSON; <code>02</code> reduces them to <code>stg_trace_courier_minute</code> (courier × minute, state ∈ <code>en_route</code> / <code>idle</code> / <code>cooldown</code> / <code>offline</code>) and does not carry raw pings forward . One reference month is kept full for the data dictionary.
Windows / OneDrive	Write to a temp path in the same folder, then atomic <code>os.replace</code> ; a retry-on-lock wrapper (<code>utils.write_parquet</code> , ~4 attempts, backoff) absorbs intermittent WinError 5/32 locks.

```
# from repo root
python -m pip install -r etl/requirements.txt
python etl/run_pipeline.py           # full build: raw -> staging -> curated -> dq_checks
python etl/run_pipeline.py --from 02 # rebuild staging + curated only
python etl/gen_data_dictionary.py    # regenerate docs/03_data_dictionary.md from disk
```

Run order inside `run_pipeline.py` :

- 1. `01_generate_raw.py` — writes `data/raw/**`
- 2. `02_clean_stage.py` — writes `data/staging/stg_*.parquet` + `data/quality/dq_fragments/stg_*.json`
- 3. `03_build_curated.py` — writes `data/curated/{dim,fact}_*. {parquet,csv}` + curated DQ fragments
- 4. `dq_checks.py` — hard assertions; assembles `data/quality/dq_report.md` ; non-zero exit on failure

3. Naming conventions

Thing	Rule	Example
Raw file	<code><source>__<entity>__<period-or-grain>.<ext></code>	<code>ordercore__orders__2026-04.csv</code>
Staging table	<code>stg_<source>_<entity></code>	<code>stg_ordercore_orders</code>
Dimension	<code>dim_<noun></code>	<code>dim_zone</code>
Fact	<code>fact_<process>_<grain></code>	<code>fact_zone_hour</code>
Surrogate key	<code><entity>_key</code> — integer, curated only	<code>zone_key</code>
Business key	<code><entity>_id</code> — preserved from source	<code>courier_id</code>
Date	<code><role>_date</code> (date) · <code><role>_date_key</code> (int YYYYMMDD)	<code>placed_date</code> , <code>placed_date_key</code>
Timestamp	<code><role>_ts</code> — tz-aware, America/Sao_Paulo in curated	<code>dropoff_ts</code>
Money	<code><measure>_brl</code> (single currency)	<code>basket_value_brl</code> , <code>courier_payout_brl</code>
Duration	<code><measure>_min</code> (minutes) · <code><measure>_s</code> (seconds)	<code>eta_min</code> , <code>assignment_latency_s</code>
Distance	<code><measure>_km</code>	<code>trip_distance_km</code>
Rate / ratio	<code><measure>_rate</code> / <code><measure>_ratio</code> (0-1)	<code>fulfillment_rate</code> , <code>idle_courier_ratio</code>
Boolean	<code>is_<x></code> / <code>has_<x></code>	<code>is_peak_block</code> , <code>on_treated_zone_day</code>
Columns	<code>snake_case</code> throughout curated	

Source timestamps are parsed to tz-aware UTC in staging, then curated derives the **Brazil-local calendar day and hour** (`America/Sao_Paulo`) and every `*_date_key` from that local day — so day-boundary, hour-block and experiment-week bucketing all land correctly, and the PayHub UTC feed (§4 #5) is reconciled with the rest.

4. Source-system inventory

Six simulated operational systems plus one static geo reference. "Export style" is what makes each one messy.

#	System (fictional)	Owns	Raw entities	Export style / quirks
1	OrderCore (consumer order backend)	Orders of record: placement, status lifecycle, basket, delivery address, restaurant	<code>orders</code> (monthly), <code>order_status_events</code> (monthly), <code>addresses</code>	Mobile client retries on flaky connections → exact & near-exact duplicate <code>orders</code> / <code>order_status_events</code> rows; <code>placed_at</code> mixes DD/MM/YYYY HH:MM and ISO-8601; <code>basket_value</code> sometimes "R\$ 62,90" , sometimes 62.9 ; address <code>bairro</code> / <code>logradouro</code> with encoding damage ; <code>delivery_zone</code> on older rows is the zone as the map stood then (see #7).
2	RiderApp (courier app backend)	Courier sessions, chosen work zone/block, dispatch offers and the accept/reject/timeout outcome, pickup/dropoff stamps	<code>courier_sessions</code> , <code>shift_zone_blocks</code> , <code>offers</code>	<code>login</code> / <code>logout</code> local time, no tz marker; <code>offers</code> is at-least-once (duplicate <code>offer_id</code>); <code>courier_id</code> free-text casing; a slice of <code>offers</code> reference a <code>courier_id</code> absent from the courier master (client cache lag).
3	Trace (location service)	Courier GPS position over time; the basis for idle vs en-route minutes	<code>location_pings</code> (NDJSON, monthly)	<code>ts</code> as Unix-epoch milliseconds ; lat/long jitter , a tail of pings hundreds of km away (device cold-start) and (<code>0</code> , <code>0</code>) nulls; ping cadence irregular (30–90 s); occasional multi-hour gaps mid-session.
4	Dispatch telemetry (<i>assignment engine log</i>)	Per-order assignment funnel: ready → assigned → picked-up → delivered, assignment latency, "no courier found"	<code>assignment_events</code>	One row per state transition; <code>no_courier_found</code> emitted then sometimes followed by a late assignment (must reconcile to a single outcome); duplicate transition rows; latency in milliseconds as text .
5	PayHub (payments & payouts)	Order payment, courier payout, tip, incentive-bonus payout	<code>order_payments</code> , <code>courier_payouts</code> , <code>bonus_payouts</code>	Entire feed in UTC with Z while everything else is Brazil-local, no offset column ; amounts as "1.234,56" / "(12,50)" (adjustment reversals in parentheses); <code>competencia</code> (settlement week) as Excel serial ; the 2026-05 payout file was delivered twice with different names.

#	System (fictional)	Owns	Raw entities	Export style / quirks
6	Boost (incentive / campaign system)	The experiment: per zone-day arm, bonus parameters, push radius, realised bonus spend	<code>incentive_campaigns</code> , <code>bonus_ledger</code>	Modern tool, mostly clean — but <code>bonus_ledger</code> is at-least-once ; ~20 <code>incentive_campaigns</code> rows are late edits (a zone-day re-labelled after midnight) that must be reconciled to the assignment fixed at 00:00; timestamps UTC <code>Z</code> .
7	GeoRef (<i>static reference, not a live feed</i>)	Zone polygons, adjacency graph, area, dispatch centroid, baseline supply-stress tier, boundary-redraw history	<code>zones.geojson</code> , <code>zone_adjacency.csv</code> , <code>zone_history.csv</code>	Two zone polygons were re-cut at window-month 9 (2025-09); <code>zones.geojson</code> is the current map, <code>zone_history.csv</code> carries the prior polygons with valid-from/to so curated can resolve the as-booked zone for older orders.

`assignment_events` (#4) is the **order-outcome source of truth** (delivered vs which cancellation); `order_status_events` (#1) is kept to cross-check and to time the customer's own cancel. Boost (#6) corresponds to the experiment itself — a designed intervention, not "neutral history".

5. Raw layer — what `01_generate_raw.py` emits

Volumes are approximate at `SEED` default, over the 20-month window.

Raw file(s)	Grain	~Rows	Key messy traits injected
<code>ordercore__orders__YYYY-MM.csv</code>	one order	~7.5M	retry duplicates (~1.5%); mixed datetime formats; <code>basket_value</code> / <code>delivery_fee</code> sometimes text; free-text <code>status</code> ; <code>delivery_zone</code> = as-booked zone label
<code>ordercore__order_status_events__YYYY-MM.csv</code>	order × status transition	~28M	at-least-once duplicates; free-text status (<code>placed</code> / <code>PLACED</code> / <code>created</code>); local time no tz
<code>ordercore__addresses__YYYY-MM.csv</code>	delivery address	~2.1M	<code>bairro</code> / <code>logradouro</code> mojibake; lat/long occasionally swapped; CEP with/without dash
<code>riderapp__courier_sessions.csv</code>	login→logout session	~1.6M	local time no tz marker; overlapping sessions on app crash+reopen; <code>courier_id</code> casing
<code>riderapp__shift_zone_blocks.csv</code>	courier × chosen zone × block	~4.4M	courier's <i>intended</i> zone (not where they ended up); free-text block labels
<code>riderapp__offers.csv</code>	dispatch offer	~14M	duplicate <code>offer_id</code> ; outcome free text (<code>accept</code> / <code>ACCEPTED</code> / <code>ignored</code> / <code>timeout</code>); ~1.5% orphan <code>courier_id</code>
<code>trace__location_pings__YYYY-MM.ndjson.gz</code>	courier ping	~380M (reduced in staging)	epoch-ms <code>ts</code> ; lat/long jitter; far-away & (<code>0,0</code>) pings; irregular cadence; mid-session gaps
<code>dispatch__assignment_events.csv</code>	order × transition	~26M	<code>no_courier_found</code> then late assignment; duplicate transitions; latency ms-as-text
<code>payhub__order_payments.csv</code>	order payment	~7.3M	UTC <code>Z</code> ; amounts as locale text; Excel-serial settlement week
<code>payhub__courier_payouts.csv</code>	courier × settlement week × order	~7.4M	UTC <code>Z</code> ; parentheses reversals; 2026-05 file duplicated
<code>payhub__bonus_payouts.csv</code>	bonus payout	~90k	at-least-once; UTC <code>Z</code> ; only exists for treated zone-days
<code>boost__incentive_campaigns.csv</code>	zone × date (peak block)	~7,300 all-window / ~840 in the experiment window	~20 post-midnight late edits to reconcile; UTC <code>Z</code> ; <code>arm</code> present only for window rows
<code>boost__bonus_ledger.csv</code>	bonus accrual line	~95k	at-least-once duplicates
<code>georef__zones.geojson</code>	zone polygon (current)	12	current map only
<code>georef__zone_adjacency.csv</code>	zone pair	~34	undirected edges listed once
<code>georef__zone_history.csv</code>	zone polygon version	~16	prior polygons for the 2 redrawn zones, valid-from/to

6. Staging layer — `02_clean_stage.py`

One `stg_*` table per raw entity (plus the derived `stg_trace_courier_minute`). Responsibilities and the DQ counter each raises:

Fix class	What happens	DQ counter(s)
Date/time parsing	Parse DD/MM/YYYY[HH:MM] , ISO-8601 (with/without TZ), Unix-epoch ms (Trace), Excel serials (PayHub settlement week).	<code>datetimes_parsed</code> , <code>epoch_ms_converted</code> , <code>excel_serials_converted</code> , <code>datetimes_unparseable</code>
Timezone	Everything → tz-aware UTC. RiderApp / OrderCore local stamps localised from <code>America/Sao_Paulo</code> ; PayHub / Boost already <code>Z</code> . Flag the assumed offset on the localised feeds.	<code>timestamps_localized</code> , <code>utc_feeds_passthrough</code>
Money parsing	Strip <code>R\$</code> , thousands <code>.</code> , decimal <code>,</code> ; <code>"(x)"</code> → <code>-x</code> ; blank → null (never 0).	<code>money_strings_parsed</code> , <code>money_negatives_from_parens</code>
GPS sanitisation	Drop pings outside the metro bounding box + <code>(0,0)</code> ; clamp jitter with a max-plausible-speed filter; resample to 1-minute courier state (en_route / idle / cooldown / offline) from ping density + session + offer state.	<code>gps_out_of_area_dropped</code> , <code>gps_null_island_dropped</code> , <code>gps_speed_clamped</code> , <code>ping_minutes_materialised</code>
Encoding repair	<code>ftfy.fix_text</code> + targeted Latin-1→UTF-8 re-decode for OrderCore addresses / restaurant names; strip BOM.	<code>mojibake_repaired</code> , <code>bom_stripped</code>
De-duplication	Drop exact/near-exact retry rows on <code>orders</code> & <code>order_status_events</code> ; dedupe <code>offer_id</code> / <code>bonus_ledger</code> / duplicate assignment transitions; drop the re-delivered PayHub 2026-05 file; keep only the reconciled (00:00-fixed) Boost campaign row per zone-day.	<code>retry_dupes_dropped</code> , <code>dupe_events_dropped</code> , <code>redelivered_file_dropped</code> , <code>campaign_late_edits_reconciled</code>
Outcome reconciliation	Collapse the <code>assignment_events</code> transition log to one outcome per order (<code>no_courier_found</code> followed by a later assignment → <code>delivered</code> /other actual outcome); latency ms-text → int seconds.	<code>order_outcomes_reconciled</code> , <code>latency_coerced</code>
Category harmonisation	Map order <code>status</code> , offer <code>outcome</code> , vehicle type, acquisition channel, cuisine, price band to controlled vocab in <code>etl/vocab/</code> . Unknowns → explicit <code>"unknown"</code> + logged.	<code>categories_normalised</code> , <code>category_unknowns</code>
Key normalisation	Trim business keys; cast to string; null-out placeholders (<code>"0"</code> , <code>" "</code> , <code>"N/A"</code>). Orphan <code>courier_id</code> left as-is for curated to sentinel.	<code>null_keys_flagged</code>
Type coercion	Enforce a declared dtype per column; failures → null + counter.	<code>type_coercion_failures</code>

Staging does **not** join across sources, does **not** compute liquidity / fulfillment / idle ratios, does **not** resolve address→zone, and does **not** drop rows for referential problems — that is curated's job, fully accounted.

7. Curated layer — `03_build_curated.py`

Kimball star. `dim_date` is built first and spans **2024-06-01 → 2027-06-30** (well past the fact window). `dim_zone` and the polygon lookup are built second (everything geo depends on them).

7.1 Dimensions

Table	Grain	Notable attributes
<code>dim_date</code>	calendar day	<code>date_key</code> , y/q/m/iso-week, <code>day_of_week</code> , <code>is_weekend</code> , <code>is_holiday_br</code> , <code>is_rainy_season</code> , <code>season_tag</code> , <code>experiment_phase</code> (pre / test / post), <code>experiment_week</code> (1–10 during 2026-04-06→2026-06-14, else null)
<code>dim_time_block</code>	hour of day (0–23)	<code>time_block_key</code> , <code>hour</code> , <code>daypart</code> (early / breakfast / lunch / afternoon / dinner_peak / late), <code>is_peak_block</code> (18:00–21:59), <code>block_label</code>
<code>dim_zone</code>	zone	<code>zone_key</code> , <code>zone_id</code> , <code>zone_name</code> , <code>macro_area</code> (centre / inner_ring / outer_ring), <code>area_km2</code> , <code>centroid_lat</code> , <code>centroid_lon</code> , <code>n_neighbours</code> , <code>baseline_supply_stress_tier</code> (short / balanced / long — fixed from months 1–15), <code>avg_trip_km_baseline</code> , <code>is_boundary_redrawn</code>
<code>dim_zone_adjacency</code>	zone × neighbour zone	<code>zone_key</code> , <code>neighbour_zone_key</code> , <code>shared_border_km</code> — the bridge for the cannibalisation analysis (adjacent = candidate to be poached from)
<code>dim_courier</code>	courier	<code>courier_key</code> , <code>courier_id</code> , <code>signup_date_key</code> , <code>signup_cohort_month</code> , <code>vehicle_type</code> (bike / moto / car), <code>home_zone_key</code> (modal drop-off, trailing 28d — descriptive only), <code>acquisition_channel</code> , <code>is_test_account</code>
<code>dim_customer</code>	customer	<code>customer_key</code> , <code>customer_id</code> , <code>signup_date_key</code> , <code>home_zone_key</code> , <code>customer_segment</code> (new / casual / regular / heavy), <code>is_test_account</code>
<code>dim_restaurant</code>	restaurant	<code>restaurant_key</code> , <code>restaurant_id</code> , <code>zone_key</code> , <code>cuisine</code> , <code>price_band</code> (R\$ / R\$\$ / R\$\$\$), <code>avg_prep_min_baseline</code>
<code>dim_incentive_campaign</code>	zone × date (peak block)	<code>campaign_key</code> , <code>zone_key</code> , <code>date_key</code> , <code>arm</code> (control / treatment), <code>bonus_brl_per_delivery</code> (0 for control), <code>push_radius_m</code> , <code>peak_block_start_hour</code> , <code>peak_block_end_hour</code> , <code>stratum</code> (zone × baseline supply-stress tier), <code>day_of_week</code>

7.2 Facts

Table	Grain	Measures / columns	Build notes
fact_order	one order	order_key , customer_key , restaurant_key , courier_key (sentinel -1 if never assigned / -2 orphan), pickup_zone_key , dropoff_zone_key_asbooked , dropoff_zone_key_current , placed_date_key , placed_ts , placed_hour , time_block_key , is_peak_block , basket_value_brl , delivery_fee_brl , outcome (delivered / cancelled_customer / cancelled_courier / cancelled_no_courier / cancelled_restaurant), eta_min (null unless delivered), assignment_latency_s , trip_distance_km , experiment_phase , on_treated_zone_day , is_adjacent_to_treated_zone_day	Retry-deduped. Address → zone by point-in-polygon against both the current and the as-of-order-date polygon set; the experiment uses _asbooked , trend geography uses _current . outcome from reconciled assignment_events .
fact_delivery	one completed delivery	delivery_key , order_key , courier_key , zone_key (= dropoff as-booked), pickup_ts , dropoff_ts , trip_distance_km , trip_min , courier_base_payout_brl , tip_brl , incentive_bonus_brl , total_payout_brl , is_peak_block , experiment_phase , on_treated_zone_day	One row per outcome = delivered . incentive_bonus_brl joined from bonus_payouts / bonus_ledger (reconciled); non-zero only on treated zone-day peak blocks.
fact_courier_shift_block	courier × zone × date × hour-block	courier_key , zone_key , date_key , time_block_key , logged_in_min , active_min (en-route), idle_min , cooldown_min , deliveries , base_payout_brl , tip_brl , bonus_brl , earnings_brl	Built from stg_trace_courier_minute + sessions + offers + deliveries. zone_key = the zone the courier's position was in for that minute-majority. The supply side of liquidity.
fact_zone_hour	zone × date × hour	zone_key , date_key , time_block_key , is_peak_block , orders_placed , orders_delivered , cancelled_customer , cancelled_courier , cancelled_no_courier , cancelled_restaurant , fulfillment_rate , eta_p50_min , eta_p90_min , assignment_latency_p90_s , available_courier_hours , demanded_delivery_hours , liquidity_ratio , active_couriers , idle_courier_hours , idle_courier_ratio , unmet_demand , is_rainy , has_local_event , experiment_arm (peak-block window rows only), on_treated_zone_day , is_adjacent_to_treated_zone_day	The liquidity spine and the report's workhorse. demanded_delivery_hours = orders_placed × the period-average service time (pickup + drive + drop). available_courier_hours = Σ fact_courier_shift_block (active + idle) for the zone-hour. liquidity_ratio = available + demanded.
fact_incentive_assignment	zone-day (peak block), experiment window	zone_key , date_key , campaign_key , arm , bonus_brl_per_delivery , stratum , day_of_week , n_adjacent_zones_treated_same_day , is_adjacent_to_treated , pre-period covariates frozen at 2026-04-05 : pre_fulfillment_rate , pre_eta_p90_min , pre_orders_placed_mean , pre_liquidity_ratio , pre_idle_courier_ratio	~840 rows. Late-edited campaign rows reconciled to the 00:00 assignment. Covariates from the 15-month baseline; used for the SRM / balance checks and variance reduction in Phase 10.
fact_experiment_zone_day	zone-day (peak block), experiment window	zone_key , date_key , arm , stratum , fulfillment_rate (primary), orders_placed , orders_delivered , eta_p50_min , eta_p90_min , cancel_rate , cancel_no_courier_rate , available_courier_hours , active_couriers , courier_earnings_per_active_hour_brl , deliveries , incentive_spend_brl , incentive_cost_per_delivered_order_brl , n_adjacent_zones_treated , is_adjacent_to_treated	~840 rows — one tidy row per unit of randomization , every metric the read-out needs pre-computed so Phase 10 goes straight to tests and the report needs almost no DAX.
fact_experiment_zone_block_week	zone × experiment-week × peak hour-block	zone_key , experiment_week , time_block_key , arm , fulfillment_rate , orders_placed , orders_delivered , eta_p90_min , available_courier_hours	The weekly × block series for the novelty / decay diagnostic and the time-of-peak cut.
fact_targets_zone_hour	zone × hour-of-day × kpi	target_value	Plan from doc 01 \$10; same basis as actuals.

7.3 Cross-cutting curated rules

- **Two zone maps.** Every order gets `dropoff_zone_key_asbooked` (polygon set valid on the order date) **and** `dropoff_zone_key_current` (today's map). The experiment and all zone-day/zone-hour facts key off **as-booked** — the incentive was assigned on the map in force at the time. Trend geography on the report uses **current**. Drift-row count reported.
- **Supply and demand in one unit.** Both `available_courier_hours` and `demanded_delivery_hours` are in courier-hours, so `liquidity_ratio` is dimensionless and comparable across zones and hours.
- **Idle vs active.** Derived from `stg_trace_courier_minute`: a logged-in minute is **active** if the courier is on an accepted offer through dropoff, `cooldown` for a short post-dropoff window, else **idle**. Idle minutes are attributed to the zone the courier was physically in.
- **Sentinel keys.** `courier_key = -1` (no courier ever assigned — the `cancelled_no_courier` path), `-2` (orphan — courier id not in master). Facts are **never dropped** for a missing dimension; the sentinel is used and counted.
- **Experiment stamping.** `arm / on_treated_zone_day / is_adjacent_to_treated_zone_day` are resolved from `fact_incentive_assignment` (via `zone_key + date_key`) and stamped onto `fact_order` and `fact_zone_hour` peak-block rows, so the report can slice liquidity by arm without touching the experiment facts.
- **Adjacency-aware spillover flags.** `is_adjacent_to_treated_zone_day` = the zone is a control zone that day **and** at least one `dim_zone_adjacency_neighbour` was treated that day. `n_adjacent_zones_treated` carries the count. These drive guardrail **G5**.
- **Local calendar** (`America/Sao_Paulo`) drives every `*_date_key`, `placed_hour` and the experiment-week bucketing; the PayHub/Boost UTC feeds are converted before keying.
- **BRL only.** Every money column is `*_brl`. No `_usd` sibling, no FX table.
- **Lapse/- style honesty.** `fact_zone_hour.fulfillment_rate` is null (not 0) for a zone-hour with `orders_placed = 0`. Rates are never computed on a zero denominator.

8. Data-quality issue catalogue (the injection list)

Eight **primary** classes — injected by `01`, handled by `02 / 03`, counted into a DQ fragment, and (where an invariant) asserted in `dq_checks.py`.

#	Issue	Injected in	Handled in	Hard check
Q01	Multiple date/time formats + Unix-epoch ms + Excel serials	ordercore, riderapp, trace (epoch-ms), payhub (serial), boost	staging datetime parser	<code>datetimes_unparseable = 0</code>
Q02	Timezone drift — PayHub & Boost in UTC <code>Z</code> , no offset, while OrderCore/RiderApp are Brazil-local with no marker	payhub, boost vs ordercore, riderapp	staging: localise the naive feeds, pass through the <code>Z</code> feeds, reconcile in curated to local day/hour	no naive timestamps in curated; peak-block bucketing exact against <code>America/Sao_Paulo</code>
Q03	Money as locale strings / parentheses negatives / symbol-in-value	ordercore <code>basket_value</code> , payhub amounts	staging money parser	no non-null money column is object dtype; <code>total_payout_brl ≥ 0</code> ; <code>basket_value_brl > 0</code> for placed orders
Q04	GPS noise & out-of-area pings — lat/long jitter, far-away cold-start pings, <code>(0,0)</code> nulls, irregular cadence	trace <code>location_pings</code>	staging GPS sanitiser (bbox + speed filter + 1-minute resample)	0 pings outside the metro bbox survive into <code>stg_trace_courier_minute</code> ; courier-minute state ∈ controlled set
Q05	Duplicate rows — mobile retry dupes, at-least-once event/offer/bonus logs, re-delivered PayHub month	ordercore, riderapp, dispatch, payhub, boost	staging de-dup (near-dup collapse / dedupe on id / drop re-delivered file)	<code>fact_order</code> unique on <code>order_id</code> ; <code>fact_delivery</code> unique on <code>delivery_id</code> ; <code>fact_experiment_zone_day</code> unique on <code>(zone_key , date_key)</code>
Q06	Missing master rows / orphan keys — <code>offers</code> & trip rows for a <code>courier_id</code> absent from the courier master	riderapp, dispatch	curated sentinel <code>courier_key = -2</code> + count	every fact FK resolves to a real or sentinel key; orphan count reported
Q07	Zone boundary redraw — current <code>zones.geojson</code> disagrees with the as-of-date polygons for 2 zones from window-month 9	georef	curated resolves both <code>_asbooked</code> and <code>_current</code> zone keys; experiment keys off <code>_asbooked</code>	every order has a non-null <code>dropoff_zone_key_asbooked</code> ; as-booked ≠ current drift-row count reported (expected in the 2 redrawn zones only)
Q08	Encoding damage (Latin-1 mojibake, BOM)	ordercore addresses & restaurant names	<code>ftfy</code> + targeted re-decode	no <code>Ã / Ä</code> mojibake sequences remain in <code>dim_restaurant.restaurant_id</code> labels or address fields feeding <code>dim_zone</code> names

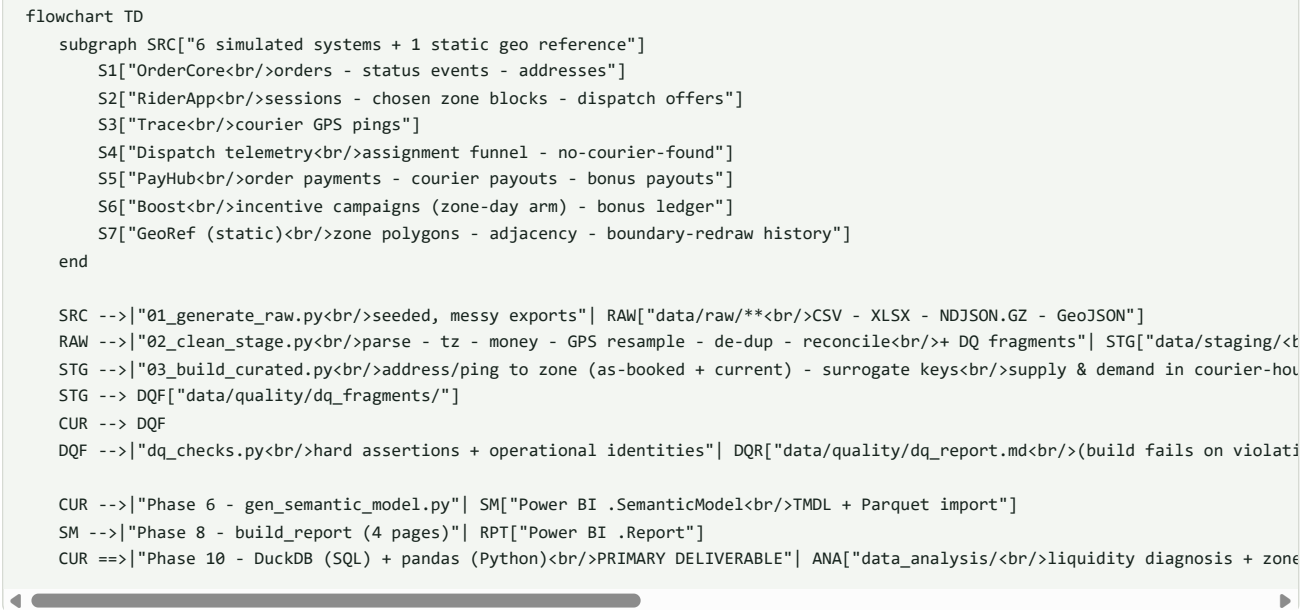
Also handled, without ceremony (minor, single counter each): null/placeholder keys → sentinels; negative or out-of-range durations/distances → clamped + counted; `no_courier_found` → late-assignment reconciliation; Boost post-midnight campaign edits → 00:00 assignment wins; offer/status/vehicle/cuisine free-text → controlled vocab.

`dq_checks.py` also asserts the **operational identities**:

- **Order conservation**, per zone-hour: `orders_placed = orders_delivered + cancelled_customer`
- `cancelled_courier + cancelled_no_courier + cancelled_restaurant` (exact).
- **Supply tie-out**: `fact_zone_hour.available_courier_hours == Σ ( active_min + idle_min ) ÷ 60` from `fact_courier_shift_block` for the same zone-hour, within 0.5%.

- **Rate sanity:** $\text{fulfillment_rate} \in [0, 1]$; $\text{eta_p50_min} \leq \text{eta_p90_min}$; $\text{liquidity_ratio} \geq 0$; $\text{idle_courier_ratio} \in [0, 1]$.
- **Delivery/payout tie-out:** $\sum \text{fact_delivery.total_payout_brl} == \sum \text{fact_courier_shift_block.earnings_brl}$ within 0.5%; $\sum \text{incentive_bonus_brl} == \text{boost reconciled bonus spend}$ within 0.5%.
- **Experiment integrity:** every $\text{fact_experiment_zone_day} / \text{fact_experiment_zone_block_week}$ key $\in \text{fact_incentive_assignment}$; treated zone-days have $\text{bonus_brl_per_delivery} > 0$, $\text{control} = 0$; per-stratum allocation ratio $\in [0.45, 0.55]$ (a *build-time* sanity check — the real SRM test is on observed data in Phase 10); pre-period covariates present for all ~840 units; fact data extends ≥ 75 days past 2026-06-14.
- **Benchmark bands:** fulfillment rate, ETA p50/p90, cancellation rate, idle-courier ratio, liquidity ratio and courier earnings per active hour all land inside their doc 01 \$9 ranges for the **pre-experiment baseline** period (the experiment arms are allowed to diverge in-window — that is the point).

9. Architecture diagram



10. Open items carried into Phase 3

#	Item	Resolution plan
B1	The true incentive effect — primary lift on fulfillment rate, per-tier heterogeneity profile (large in supply-short zones, ≈ 0 in supply-long), novelty-decay shape, guardrail impacts (ETA p90 down, no-courier cancels down, courier earnings up), and the neighbour-zone cannibalisation term	Decided in Phase 3 , encoded in <code>config.py</code> as tunable effect sizes + noise; kept out of docs 00/01; detected blind in Phase 10. Design intent: a real, non-trivial but not-obvious pooled lift, concentrated in short zones; a modest early novelty bump that decays over the 10 weeks; guardrail G1 (incentive cost per incremental delivered order) deliberately lands near the contribution-margin threshold so it needs a careful read; cannibalisation real but partial (metro net lift $\approx 70\text{--}85\%$ of the summed local lift).
B2	Courier supply model — session start/stop, zone choice, and the elasticity of zone presence to expected earnings and to the bonus	Seeded discrete-choice model over zones per block; the bonus shifts the utility of a treated zone; the elasticity parameter is the structural quantity Phase 10 must recover. Calibrate so baseline supply-short zone-hours $\approx 8\text{--}12\%$ of peak.
B3	Row volumes vs targets (~7.5M orders; ~380M raw pings) and the GPS reduction step	First regeneration pass calibrates population and activity rates; confirm <code>stg_trace_courier_minute</code> reduces pings $\sim 100\times$ and the pipeline runs end-to-end in a sane time on this box.
B4	Benchmark-band calibration (fulfillment, ETA p50/p90, cancellation rate, idle ratio, liquidity ratio, courier earnings/hr)	Several regeneration passes until the pre-experiment baseline sits mid-band for every doc 01 \$9 KPI.
B5	Zone geometry — 12 polygons, adjacency graph, dispatch centroids, baseline supply-stress tier assignment, and the 2 mid-window redraws	Hand-authored <code>zones.geojson</code> (a stylised 12-cell tiling of a metro) + <code>zone_history.csv</code> ; tiers assigned from the calibrated baseline liquidity, not by hand.
B6	BR calendar, rainy-season curve, local-event calendar (football / shows by zone)	Static lookups in <code>et1/vocab/</code> (<code>br_calendar.csv</code> , <code>rainy_season.csv</code> , <code>local_events.csv</code>); seasonal + weather multipliers in <code>config.py</code> .
B7	Exact experiment sizing — eligible zone-days, per-stratum counts, day-of-week balancing	Generator targets 12 zones $\times$ ~70 window-days; assignment draws 50/50 within each (zone $\times$ supply-stress tier) cell, balanced on weekday/weekend; confirm ≈ 420 zone-days/arm and $\geq \sim 140$ in the short tier per arm (doc 01 \$5.4).

End of Phase 2 deliverable.

SwiftBite Delivery — Data Dictionary

Auto-generated from `data/curated/` on 2026-09-10 · SEED=20260910. Do not edit by hand — run `python etl/gen_data_dictionary.py`.

15 curated tables.

dim_courier · 1,400 rows

One row per courier. Signup cohort, vehicle, acquisition channel, modal home zone. courier_key -1 = no courier assigned, -2 = orphan.

column	dtype	non-null	distinct	example
courier_key	int64	100.0%	1,400	1
courier_id	str	100.0%	1,400	C100000
signup_date_key	int32	100.0%	590	20250420
signup_cohort_month	str	100.0%	23	2025-04
vehicle_type	str	100.0%	3	bike
acquisition_channel	str	100.0%	4	referral
home_zone_key	int64	100.0%	12	1
is_test_account	bool	100.0%	1	False

dim_customer · 9,000 rows

One row per ordering customer, with a coarse activity segment.

column	dtype	non-null	distinct	example
customer_key	int64	100.0%	9,000	1
customer_id	str	100.0%	9,000	U200000
customer_segment	str	100.0%	4	new
home_zone_key	int64	100.0%	12	8

dim_date · 1,125 rows

Calendar spine (2024-06 → 2027-06). Carries experiment_phase / experiment_week, BR holidays, rainy-season flag.

column	dtype	non-null	distinct	example
date	datetime64[ms]	100.0%	1,125	2024-06-01 00:00:00
date_key	int32	100.0%	1,125	20240601
year	int32	100.0%	4	2024
quarter	int32	100.0%	4	2
month	int32	100.0%	12	6
month_name	str	100.0%	12	Jun
iso_week	UInt32	100.0%	53	22
day_of_week	int32	100.0%	7	5
is_weekend	bool	100.0%	2	True
is_holiday_br	bool	100.0%	2	False
is_rainy_season	bool	100.0%	2	False
experiment_phase	str	100.0%	3	pre
experiment_week	float64	6.2%	10	1.0

dim_incentive_campaign · 840 rows

One row per peak-eligible zone-day in the 10-week experiment window: arm, bonus, stratum (zone × stress tier), day-of-week.

column	dtype	non-null	distinct	example
campaign_key	int64	100.0%	840	1
zone_key	int64	100.0%	12	1
date_key	int64	100.0%	70	20260406
arm	str	100.0%	2	control
bonus_br1_per_delivery	float64	100.0%	2	0.0
push_radius_m	int64	100.0%	1	2000
peak_block_start_hour	int64	100.0%	1	18
peak_block_end_hour	int64	100.0%	1	22
stratum	str	100.0%	12	Z01
day_of_week	int32	100.0%	7	0

dim_restaurant • 1,200 rows

One row per restaurant, its zone, cuisine and price band.

column	dtype	non-null	distinct	example
restaurant_key	int64	100.0%	1,200	1
restaurant_id	str	100.0%	1,200	R300000
zone_key	int64	100.0%	12	9
cuisine	str	100.0%	7	burger
price_band	str	100.0%	3	R\$

dim_time_block • 24 rows

One row per hour of day (0–23) with daypart and is_peak_block (18:00–21:59, the experiment's block).

column	dtype	non-null	distinct	example
time_block_key	int64	100.0%	24	0
hour	int64	100.0%	24	0
daypart	str	100.0%	6	early
is_peak_block	bool	100.0%	2	False
block_label	str	100.0%	24	00:00–01:00

dim_zone • 12 rows

The 12 delivery zones. baseline_supply_stress_tier (short/balanced/long) is derived from months 1–15 liquidity; is_boundary_redrawn flags the two zones re-cut at window-month 9.

column	dtype	non-null	distinct	example
zone_key	int64	100.0%	12	1
zone_id	str	100.0%	12	Z01
zone_name	str	100.0%	12	Centro
macro_area	str	100.0%	3	centre
area_km2	float64	100.0%	12	7.5
centroid_lat	float64	100.0%	6	-19.92
centroid_lon	float64	100.0%	5	-43.96
n_neighbours	int64	100.0%	4	5
baseline_supply_stress_tier	str	100.0%	3	long

column	dtype	non-null	distinct	example
avg_trip_km_baseline	float64	100.0%	3	2.4
is_boundary_redrawn	bool	100.0%	2	False

dim_zone_adjacency • 42 rows

Undirected zone adjacency as a bridge table — the basis for the neighbour-zone cannibalisation analysis.

column	dtype	non-null	distinct	example
zone_key	int64	100.0%	12	1
neighbour_zone_key	int64	100.0%	12	2

fact_courier_shift_block • 204,123 rows

Courier × zone × date × hour: logged-in / active / idle / cooldown minutes, deliveries, earnings. The supply side of liquidity.

column	dtype	non-null	distinct	example
courier_key	int64	100.0%	1,400	830
zone_key	int64	100.0%	12	2
date_key	int64	100.0%	608	20250101
time_block_key	int64	100.0%	22	0
logged_in_min	int64	100.0%	45	44
active_min	int64	100.0%	46	0
idle_min	int64	100.0%	57	38
cooldown_min	int64	100.0%	4	3
deliveries	int64	100.0%	16	0
base_payout_br1	Float64	100.0%	7,654	0.0
tip_br1	float64	100.0%	1	0.0
bonus_br1	float64	100.0%	10	0.0
earnings_br1	Float64	100.0%	7,907	0.0
is_peak_block	bool	100.0%	2	False

fact_delivery • 351,577 rows

One row per delivered order: courier, trip, payout split (base / tip / incentive bonus / total).

column	dtype	non-null	distinct	example
delivery_key	int64	100.0%	351,577	1
order_key	int64	100.0%	351,577	1
courier_key	int64	100.0%	1,400	859
zone_key	int64	100.0%	12	2
date_key	int32	100.0%	608	20250101
time_block_key	int32	100.0%	24	0
dropoff_ts	datetime64[ns, UTC]	100.0%	328,188	2025-01-01 03:31:00+00:00
trip_min	float64	100.0%	1,647	29.975
trip_distance_km	float64	100.0%	954	1.7
courier_base_payout_br1	float64	100.0%	954	8.36
tip_br1	float64	100.0%	1,191	0.0
incentive_bonus_br1	float64	100.0%	2	0.0
total_payout_br1	float64	100.0%	1,913	8.36

column	dtype	non-null	distinct	example
is_peak_block	bool	100.0%	2	False
experiment_phase	str	100.0%	3	pre
on_treated_zone_day	bool	100.0%	2	False

fact_experiment_zone_block_week • 480 rows

Zone × experiment week × peak hour-block — the weekly series for the novelty / decay diagnostic.

column	dtype	non-null	distinct	example
zone_key	int64	100.0%	12	1
experiment_week	float64	100.0%	10	1.0
time_block_key	int32	100.0%	4	18
orders_placed	int64	100.0%	58	44
orders_delivered	int64	100.0%	57	44
eta_p90_min	float64	100.0%	475	30.976
available_courier_hours	float64	100.0%	409	21.017
fulfillment_rate	float64	100.0%	95	1.0
date_key	int64	100.0%	1	20260406
arm	str	100.0%	2	control

fact_experiment_zone_day • 840 rows

One tidy row per unit of randomization (~840): fulfillment_rate (primary), ETA p50/p90, cancel & no-courier rate, courier earnings/active-hr, incentive spend and cost per delivered order, adjacency-to-treated.

column	dtype	non-null	distinct	example
zone_key	int64	100.0%	12	1
date_key	int32	100.0%	70	20260406
orders_placed	int64	100.0%	45	21
orders_delivered	int64	100.0%	42	21
eta_p50_min	float64	100.0%	724	26.838
eta_p90_min	float64	100.0%	824	30.25
cancelled_no_courier	int64	100.0%	4	0
available_courier_hours	float64	100.0%	519	9.717
active_couriers	float64	100.0%	8	4.0
fulfillment_rate	float64	100.0%	65	1.0
cancel_rate	float64	100.0%	65	0.0
cancel_no_courier_rate	float64	100.0%	39	0.0
arm	str	100.0%	2	control
stratum	str	100.0%	12	Z01
courier_earnings_per_active_hour_br1	float64	100.0%	838	31.16
incentive_spend_br1	float64	100.0%	49	0.0
deliveries	int64	100.0%	42	21
incentive_cost_per_delivered_order_br1	float64	100.0%	192	0.0
n_adjacent_zones_treated	int64	100.0%	6	1
is_adjacent_to_treated	bool	100.0%	2	True

fact_incentive_assignment • 840 rows

One row per experiment zone-day (~840). Arm, bonus, stratum, adjacency-to-treated flags, and pre-period (months 1–15) covariates frozen for the balance checks.

column	dtype	non-null	distinct	example
campaign_key	int64	100.0%	840	1
zone_key	int64	100.0%	12	1
date_key	int64	100.0%	70	20260406
arm	str	100.0%	2	control
bonus_brl_per_delivery	float64	100.0%	2	0.0
push_radius_m	int64	100.0%	1	2000
peak_block_start_hour	int64	100.0%	1	18
peak_block_end_hour	int64	100.0%	1	22
stratum	str	100.0%	12	Z01
day_of_week	int32	100.0%	7	0
pre_fulfillment_rate	float64	100.0%	12	0.981
pre_eta_p90_min	float64	100.0%	12	33.201
pre_orders_placed_mean	float64	100.0%	12	6.56
pre_liquidity_ratio	float64	100.0%	12	1.281
pre_idle_courier_ratio	float64	100.0%	12	0.329
n_adjacent_zones_treated	int64	100.0%	6	1
is_adjacent_to_treated	bool	100.0%	2	True

fact_order • 368,658 rows

One row per placed order. Dual zone resolution (dropoff_zone_key_asbooked vs _current); outcome, ETA, assignment latency; experiment flags (on_treated_zone_day, is_adjacent_to_treated_zone_day). courier_key filled for delivered orders.

column	dtype	non-null	distinct	example
order_key	int64	100.0%	368,658	1
order_id	str	100.0%	368,658	O5000001
customer_key	int64	100.0%	9,000	8760
restaurant_key	int64	100.0%	1,200	159
pickup_zone_key	int64	100.0%	12	2
dropoff_zone_key_asbooked	int64	100.0%	12	2
dropoff_zone_key_current	int64	100.0%	12	2
placed_date_key	int32	100.0%	608	20250101
placed_ts	datetime64[ns, UTC]	100.0%	343,196	2025-01-01 03:31:00+00:00
placed_hour	int32	100.0%	24	0
time_block_key	int32	100.0%	24	0
is_peak_block	bool	100.0%	2	False
basket_value_brl	Float64	100.0%	23,132	38.89
delivery_fee_brl	Float64	100.0%	1,381	8.25
outcome	str	100.0%	5	delivered
eta_min	float64	95.4%	1,647	54.5
assignment_latency_s	Float64	100.0%	581	258.0
trip_distance_km	float64	100.0%	958	1.7
experiment_phase	str	100.0%	3	pre
on_treated_zone_day	bool	100.0%	2	False

column	dtype	non-null	distinct	example
is_adjacent_to_treated_zone_day	bool	100.0%	2	False
courier_key	int64	100.0%	1,401	859

fact_zone_hour · 117,997 rows

Zone × date × hour — the liquidity spine. Orders placed/delivered/cancelled by cause, fulfillment_rate, ETA p50/p90, available vs demanded courier-hours, liquidity_ratio, idle_courier_ratio, unmet_demand, rain / event flags, experiment_arm.

column	dtype	non-null	distinct	example
zone_key	int64	100.0%	12	1
date_key	int32	100.0%	608	20250101
hour	int32	100.0%	24	1
orders_placed	int64	100.0%	23	1
orders_delivered	int64	100.0%	23	1
cancelled_customer	int64	100.0%	5	0
cancelled_courier	int64	100.0%	4	0
cancelled_no_courier	int64	100.0%	4	0
cancelled_restaurant	int64	100.0%	3	0
assignment_latency_p90_s	Float64	100.0%	6,805	231.0
eta_p50_min	float64	98.9%	2,575	38.6
eta_p90_min	float64	98.9%	14,585	38.6
time_block_key	int32	100.0%	24	1
is_peak_block	bool	100.0%	2	False
fulfillment_rate	float64	100.0%	56	1.0
unmet_demand	int64	100.0%	6	0
demanded_delivery_hours	float64	100.0%	54	0.433
available_courier_hours	float64	100.0%	530	0.617
idle_courier_hours	float64	100.0%	446	0.0
active_couriers	float64	100.0%	10	1.0
liquidity_ratio	float64	100.0%	5,297	1.423
idle_courier_ratio	float64	95.6%	9,831	0.0
is_rainy	bool	100.0%	2	False
has_local_event	bool	100.0%	2	False
experiment_arm	str	100.0%	3	
on_treated_zone_day	bool	100.0%	2	False
is_adjacent_to_treated_zone_day	bool	100.0%	2	False

SwiftBite Delivery — ETL Pipeline

Document status: Phase 4 deliverable. **Depends on:** 02_data_architecture.md . **Describes:** the code under etl/ — what each script does, in what order, and the guarantees it gives.

PT. Este documento descreve o pipeline etl/ já implementado: os quatro scripts (01_generate_raw → 02_clean_stage → 03_build_curated → dq_checks), o que cada um garante, e como rodar. Um único SEED em config.py torna tudo reproduzível byte-a-byte.

1. Run

```
python -m pip install -r etl/requirements.txt
export SB_DATA_DIR="$LOCALAPPDATA/Temp/sb_data" # optional; keeps the big layers out of OneDrive
python etl/run_pipeline.py # 01 → 02 → 03 → dq_checks
python etl/run_pipeline.py --from 02 # skip regeneration
python etl/gen_data_dictionary.py # regenerate docs/03 from data/curated/
```

run_pipeline.py imports each step module and calls its main() in order, timing each. dq_checks exits non-zero on any failed **hard** check.

Determinism. config.SEED seeds utils.rng(name), which derives one independent numpy Generator per named stream ("demand", "supply", "clear", "emit_orders", ...) so adding a draw in one place does not shift draws elsewhere. Same seed → byte-identical data/raw/ → identical data/curated/.

Idempotency. Each script rebuilds its own layer. On Windows/OneDrive utils._atomic_write retries on lock (WinError 5/32) and falls back to an in-place write; pointing SB_DATA_DIR outside the synced tree avoids the locks entirely.

2. 01_generate_raw.py — the marketplace simulator

A seeded simulation, **vectorised at the zone-hour grain**, then serialised to messy source exports.

2.1 World

gen_georef writes the 12-zone grid, the adjacency graph and the boundary-redraw history. gen_couriers / gen_customers / gen_restaurants write the masters (with id-casing noise and mojibake injected).

2.2 Demand

For each zone × day × hour: orders ~ Poisson(λ) where λ = ORDERS_PER_DAY_METRO_BASE × day-of-week × month-seasonality × rainy-season × per-day-rain × local-event, split across zones by a fixed demand share and across hours by HOUR_SHAPE (a lunch + dinner double peak).

2.3 Supply

Metro courier-hours for the hour = demanded_courier_hours × BASE_SUP_RATIO (× rainy / per-day-rain penalties, × a slow 2026 decline). Distributed across zones by SUPPLY_W = courier-hour-demand-share × exp(0.20 · attractiveness_z) — deliberately **not** proportional to demand, so central zones run long and peripheral zones run short. This is the structural source of the liquidity mismatch the experiment targets.

2.4 Clearing

liquidity_ratio L = available_courier_hours / demanded_courier_hours. Fulfillment rate, ETA p50, the p90/p50 spread and assignment latency are smooth piecewise functions of L (FULFILL_AT_L, ETA_P50_AT_L, ...). Delivered count ~ Binomial(orders, fulfillment(L)); the non-delivered remainder is split by cause (no_courier a minority, then customer, courier, restaurant). Per-order ETA and latency are log-normal around the zone-hour central values.

2.5 The experiment (peak hours only, experiment window only)

assign_experiment splits every peak-eligible zone-day 50/50 treatment/control, stratified by zone × baseline stress tier and balanced on weekday/weekend. On a **treated** zone-day the peak-block available_courier_hours is multiplied by INCENTIVE_EFFECT.supply_response_mult_by_tier (short 1.34 / balanced 1.15 / long 1.04), amplified in week 1 by a novelty factor that decays with t = 2.5 weeks, with lognormal per-zone-day noise. cannibalisation_share = 0.28 of the added courier-hours is then subtracted from **adjacent control zones** on the same day — the spatial spillover the Phase-10 read-out must recover. None of these numbers appear outside config.py.

2.6 Messy exports (the DQ injection — see 08_data_quality.md)

Orders written monthly with mixed DD/MM/YYYY HH:MM / ISO timestamps, money as text ("R\$ 62,90"), ~1.5 % retry duplicates; status/offer/assignment logs at-least-once; PayHub & Boost in UTC Z; GPS reference (last 7 days) as epoch-ms NDJSON with jitter, far-away and null-island pings; orphan courier ids; the zone boundary redraw carried in the as-booked label; mojibake in address / restaurant text; a re-delivered PayHub month; ~20 post-midnight Boost campaign edits.

3. 02_clean_stage.py — raw → stg_*

One typed table per raw entity. Responsibilities:

Fix	How	Counter(s)
Datetime	split parser: <code>^\d{4}-</code> → ISO fast path, else <code>%d/%m/%Y %H:%M</code> ; local feeds localised <code>America/Sao_Paulo</code> → UTC; PayHub/Boost parsed as UTC	<code>datetimes_unparseable</code> , <code>utc_datetimes_unparseable</code>
GPS ping → minute	drop null-island + out-of-bbox, resample to courier × minute with a <code>moving / idle</code> state from inter-ping displacement (7-day reference only)	<code>gps_null_island_dropped</code> , <code>gps_out_of_area_dropped</code> , <code>ping_minutes_materialised</code>
Money	strip <code>R\$ / thousands . ; , → . ; (x) → -x</code>	<code>money_strings_parsed</code> , <code>money_negatives_from_parens</code>
Encoding	<code>ftfy.fix_text</code> on address / restaurant text	<code>mojibake_repaired</code>
De-dup	exact retry rows, at-least-once event/offer/bonus rows, the re-delivered PayHub month	<code>retry_dupes_dropped</code> , <code>dupe_events_dropped</code> , <code>redelivered_rows_dropped</code>
Reconcile	Boost post-midnight edits → earliest (<code>00:00</code> -fixed) row per zone-day; dispatch funnel latency ms-text → seconds	<code>campaign_late_edits_reconciled</code> , <code>latency_coerced</code>
Category	<code>status</code> , <code>offer outcome</code> → controlled vocab in <code>etl/vocab/</code>	<code>category_unknowns</code>

Staging performs **no** cross-source joins, address→zone resolution, or metric logic.

4. 03_build_curated.py — stg_* → the star

- **dim_date** (day grain, unique `date_key`), **dim_time_block** (24 hours), **dim_zone** (+ tier, filled after `fact_zone_hour`), **dim_zone_adjacency** bridge, **dim_courier / dim_customer / dim_restaurant**, **dim_incentive_campaign**.
- **Zone resolution.** Every order gets `dropoff_zone_key_asbooked` (the messy label, which carries the pre-redraw swaps) **and** `dropoff_zone_key_current` (recovered as the nearest zone centroid to the — de-swapped — address point). All zone-hour / experiment facts key off `_current`; the redraw predates the experiment window by 7 months, so within the experiment the two agree.
- **fact_order** (order grain) — outcome from the reconciled dispatch funnel, assignment latency joined from dispatch, `courier_key` filled for delivered orders from the zone-hour courier pool.
- **fact_courier_shift_block** (courier × zone × date × hour) — `logged_in` from the shift blocks, `present` from the GPS dwell rollup, deliveries split from PayHub payouts by present-minute share, `active = deliveries × service_min`, `idle = present - active - cooldown`, earnings split from payouts, bonus joined direct.
- **fact_delivery** (delivered order) — courier drawn from the same zone-hour's blocks weighted by their deliveries; payout = base (`6.4 + 1.15·km`) + tip + incentive bonus.
- **fact_zone_hour** (zone × date × hour) — the liquidity spine. Demand side aggregated from `fact_order`; supply side from `fact_courier_shift_block` (`available_courier_hours = Σ(active+idle)/60`); `liquidity_ratio`, `idle_courier_ratio`, `unmet_demand`, ETA p50/p90, cancellation split, rain / event flags, and the experiment `arm / on_treated_zone_day / is_adjacent_to_treated_zone_day` for peak-block window rows.
- **Baseline stress tier** — mean `liquidity_ratio` per zone over months 1–15, classified `< 0.93` short / `> 1.13` long / else balanced. Written onto `dim_zone`.
- **fact_incentive_assignment** (zone-day, ~840) — arm, bonus, stratum, adjacency counts, and the frozen pre-period covariates (`pre_fulfillment_rate`, `pre_eta_p90_min`, `pre_liquidity_ratio`, ...) for Phase-10 balance checks.
- **fact_experiment_zone_day** (~840) — one tidy row per unit of randomization: the primary metric and every guardrail, pre-aggregated over the peak block.
- **fact_experiment_zone_block_week** — the weekly × block series for the novelty cut.

5. dq_checks.py

Hard assertions (build fails on violation): referential integrity, `fact_order / fact_delivery` key uniqueness, every experiment zone-day present in `fact_incentive_assignment`, **order conservation** (`placed = delivered + Σ cancels`), rate sanity (`fulfillment ∈ [0,1]`), `eta_p50 ≤ eta_p90`, `idle_ratio ∈ [0,1]`), treated bonus > 0 / control bonus = 0, ≥ 75 days of post-experiment data.

Soft checks (reported to `dq_report.md`, not fatal): 50/50 allocation per stratum, non-degenerate stress tiers, and **benchmark bands** on the pre-experiment baseline — fulfillment, ETA p50/p90, cancellation rate, idle ratio, courier earnings per active hour. It also prints a **naïve** experiment read (fulfillment by arm and by tier, cost per delivered order) as a smoke test — the real read-out is Phase 10.

Current run: **13/13 hard pass · 0 soft warnings** (see `data/quality/dq_report.md`).

6. Performance notes

Full pipeline ≈ 6.5 min on the dev box (`01` ≈ 70 s, `02` ≈ 35 s, `03` ≈ 4.5 min, `dq` instant). The simulator is vectorised at the zone-hour grain (a per-order Python loop was ~3× slower); `02` uses split ISO/BR datetime fast paths (pandas `format="mixed"` was a 3-minute trap); `03`'s courier→delivery

assignment is a per-zone-hour **choice** call, not a per-order loop.

End of Phase 4 deliverable (ETL).

SwiftBite Delivery — Data Model

Document status: Phase 4 deliverable. **Depends on:** 02_data_architecture.md , 03_data_dictionary.md (auto-generated). Describes the curated star as it will be imported into the Power BI semantic model (Phase 6).

PT. O star curado: matriz de barramento (fatos × dimensões), grão e medidas de cada fato, atributos de cada dimensão, e as relações físicas para o modelo semântico.

1. Bus matrix (facts × dimensions)

Fact ↓ / Dim →	date	time_block	zone	courier	customer	restaurant	incentive_campaign
fact_order	•	•	• (dropoff current; +asbooked, pickup inactive)	•	•	•	(via zone+date)
fact_delivery	• (dropoff_ts)	•	•	•	(via order)	(via order)	(via zone+date)
fact_courier_shift_block	•	•	•	•			(via zone+date)
fact_zone_hour	•	•	•				(via zone+date)
fact_incentive_assignment	•		•				•
fact_experiment_zone_day	•		•				•
fact_experiment_zone_block_week	•	•	•				•

• = direct relationship; (via ...) = reachable through another table, not a physical relationship. Every fact carries date_key (int YYYYMMDD) → dim_date[date_key] (day grain, unique) and, where relevant, time_block_key (0–23) → dim_time_block.

2. Fact tables — grain & measures

Fact	Grain	Additive measures	Non-additive / semi-additive
fact_order	one placed order	basket_value_brl , delivery_fee_brl , count	outcome , eta_min (delivered only), assignment_latency_s , trip_distance_km , is_peak_block , on_treated_zone_day , is_adjacent_to_treated_zone_day
fact_delivery	one delivered order	trip_min , trip_distance_km , courier_base_payout_brl , tip_brl , incentive_bonus_brl , total_payout_brl	on_treated_zone_day , experiment_phase
fact_courier_shift_block	courier × zone × date × hour	logged_in_min , active_min , idle_min , cooldown_min , deliveries , base_payout_brl , tip_brl , bonus_brl , earnings_brl	is_peak_block
fact_zone_hour	zone × date × hour	orders_placed , orders_delivered , cancelled_{customer,courier,no_courier,restaurant} , unmet_demand , demanded_delivery_hours , available_courier_hours , idle_courier_hours	fulfillment_rate , eta_p50_min , eta_p90_min , assignment_latency_p90_s , liquidity_ratio , idle_courier_ratio , active_couriers , is_rainy , has_local_event , experiment_arm
fact_incentive_assignment	experiment zone-day (~840)	count	arm , bonus_brl_per_delivery , stratum , day_of_week , n_adjacent_zones_treated , is_adjacent_to_treated , pre_fulfillment_rate , pre_eta_p90_min , pre_orders_placed_mean , pre_liquidity_ratio , pre_idle_courier_ratio
fact_experiment_zone_day	experiment zone-day (~840)	orders_placed , orders_delivered , deliveries , cancelled_no_courier , available_courier_hours , incentive_spend_brl	fulfillment_rate (primary), eta_p50_min , eta_p90_min , cancel_rate , cancel_no_courier_rate , courier_earnings_per_active_hour_brl , incentive_cost_per_delivered_order_brl , active_couriers , arm , stratum ,

Fact	Grain	Additive measures	Non-additive / semi-additive
			n_adjacent_zones_treated , is_adjacent_to_treated
fact_experiment_zone_block_week	zone × experiment week × peak hour- block	orders_placed , orders_delivered , available_courier_hours	fulfillment_rate , eta_p90_min , arm

fact_experiment_zone_day and fact_incentive_assignment share the (zone_key , date_key) key 1:1 — assignment carries the design + frozen covariates, the outcome table carries the realised metrics. Phase 10 joins them; the report reads the outcome table directly.

3. Dimensions — key attributes

Dimension	Key	Attributes used in the report
dim_date	date_key (int YYYYMMDD)	year , quarter , month , month_name , iso_week , day_of_week , is_weekend , is_holiday_br , is_rainy_season , experiment_phase , experiment_week
dim_time_block	time_block_key (hour 0-23)	hour , daypart , is_peak_block , block_label
dim_zone	zone_key	zone_id , zone_name , macro_area , area_km2 , centroid_lat/lon , n_neighbours , baseline_supply_stress_tier , avg_trip_km_baseline , is_boundary_redrawn
dim_zone_adjacency	(zone_key , neighbour_zone_key)	bridge — feeds the cannibalisation / spillover measures
dim_courier	courier_key	signup_cohort_month , vehicle_type , acquisition_channel , home_zone_key , is_test_account
dim_customer	customer_key	customer_segment , home_zone_key
dim_restaurant	restaurant_key	zone_key , cuisine , price_band
dim_incentive_campaign	campaign_key (also unique on zone_key + date_key)	arm , bonus_brl_per_delivery , push_radius_m , peak_block_start_hour , peak_block_end_hour , stratum , day_of_week

Sentinel keys. courier_key = -1 (no courier ever assigned — the cancelled_no_courier path), -2 (orphan courier id not in the master). Facts are never dropped for a missing dimension row.

4. Relationships

From (fact)	Column	To	Column	Card.	Cross-filter	Active
fact_order	placed_date_key	dim_date	date_key	*:1	single	yes
fact_order	time_block_key	dim_time_block	time_block_key	*:1	single	yes
fact_order	dropoff_zone_key_current	dim_zone	zone_key	*:1	single	yes
fact_order	dropoff_zone_key_asbooked	dim_zone	zone_key	*:1	single	inactive
fact_order	pickup_zone_key	dim_zone	zone_key	*:1	single	inactive
fact_order	customer_key	dim_customer	customer_key	*:1	single	yes
fact_order	restaurant_key	dim_restaurant	restaurant_key	*:1	single	yes
fact_order	courier_key	dim_courier	courier_key	*:1	single	yes
fact_delivery	order_key	fact_order	order_key	1:1	single	yes (or merged)
fact_delivery	courier_key	dim_courier	courier_key	*:1	single	yes
fact_delivery	zone_key	dim_zone	zone_key	*:1	single	yes
fact_courier_shift_block	courier_key	dim_courier	courier_key	*:1	single	yes
fact_courier_shift_block	zone_key	dim_zone	zone_key	*:1	single	yes
fact_courier_shift_block	date_key / time_block_key	dim_date / dim_time_block	key	*:1	single	yes
fact_zone_hour	zone_key	dim_zone	zone_key	*:1	single	yes

From (fact)	Column	To	Column	Card.	Cross-filter	Active
fact_zone_hour	date_key / time_block_key	dim_date / dim_time_block	key	*:1	single	yes
fact_incentive_assignment	campaign_key	dim_incentive_campaign	campaign_key	*:1	single	yes
fact_incentive_assignment	zone_key / date_key	dim_zone / dim_date	key	*:1	single	yes
fact_experiment_zone_day	zone_key / date_key	dim_zone / dim_date	key	*:1	single	yes
fact_experiment_zone_block_week	zone_key / date_key / time_block_key	dim_zone / dim_date / dim_time_block	key	*:1	single	yes

dim_zone_adjacency is a **bridge**: dim_zone[zone_key] 1 : * dim_zone_adjacency[zone_key] , and dim_zone_adjacency[neighbour_zone_key] is used by measures (not a physical relationship) to sum a metric over a zone's neighbours for the spillover guardrail.

fact_order carries three zone roles; only **dropoff (current)** is the active relationship. dropoff (as-booked) and pickup are inactive relationships activated with USERELATIONSHIP inside specific measures (as-booked drift; pickup-zone supply).

```
erDiagram
    dim_date ||--o{ fact_order : date
    dim_time_block ||--o{ fact_order : hour
    dim_zone ||--o{ fact_order : "dropoff (current)"
    dim_customer ||--o{ fact_order : places
    dim_restaurant ||--o{ fact_order : from
    dim_courier ||--o{ fact_order : delivers
    fact_order ||--|| fact_delivery : "delivered"
    dim_zone ||--o{ fact_zone_hour : ""
    dim_date ||--o{ fact_zone_hour : ""
    dim_time_block ||--o{ fact_zone_hour : ""
    dim_courier ||--o{ fact_courier_shift_block : ""
    dim_zone ||--o{ fact_courier_shift_block : ""
    dim_incentive_campaign ||--|| fact_incentive_assignment : "zone-day"
    fact_incentive_assignment ||--|| fact_experiment_zone_day : "zone-day"
    dim_zone ||--o{ fact_experiment_zone_day : ""
    dim_zone ||--o{ dim_zone_adjacency : "bridge"
```

5. Measure groups (Phase 7 preview)

Marketplace Health (fulfillment, ETA p50/p90, cancellation by cause, assignment latency) · Liquidity (available vs demanded courier-hours, liquidity ratio, idle-courier ratio, unmet demand, spatial-mismatch index) · Supply & Couriers (active couriers, CAC, retention, deliveries/active-hour) · Courier Economics (earnings/active-hour, utilisation, contribution margin, incentive spend, **incentive cost per incremental delivered order**) · **Experiment** (fulfillment by arm + lift + cluster-robust CI + p + RI p, weekly series, balance, guardrails G1–G5, effect by tier + interaction, neighbour-zone spillover, cannibalisation-adjusted net) · Time Intelligence (WoW, trailing-4-week, YoY context) · Targets vs Plan · a dynamic Exec Insight narrative measure.

End of Phase 4 deliverable (Data Model).

SwiftBite Delivery — DAX Measure Library

Auto-generated from `etl/pbi_measures.py` on 2026-09-10 by `etl/gen_dax_doc.py`. Do not edit by hand.

71 measures in the `Measures` table, injected into the semantic model by `powerbi/gen_semantic_model.py`.

Conventions

- All money is **BRL** — no currency slicer.
- The report's date grain is the day; `dim_date[date_key]` is the calendar key on every fact. Time-of-day comes from `dim_time_block` (`is_peak_block = 18:00–21:59`).
- **ETA p50 / p90** are stored per zone-hour; the measures aggregate them as an order-weighted mean of the per-zone-hour percentile — a close approximation. The exact distribution-level percentile is a Phase-10 artifact, not a live measure.
- Experiment measures read `fact_experiment_zone_day` (one row per randomised zone-day) and `fact_incentive_assignment` (design + frozen pre-period covariates); `arm` / `stratum` are text columns filtered directly.
- The live experiment CI is a **two-sample normal interval on the zone-day means**. The cluster-robust (by zone) SE, the randomization-inference p-value and the formal arm×tier interaction test are **Phase-10 deliverables**, not measures.
- **ETA, cancellation rate, no-courier rate, idle-courier ratio and incentive cost per incremental order are lower-is-better** — their `vs Target` measures are good when negative.

01 Marketplace Health

Orders Placed

format `#,0`

```
SUM ( fact_zone_hour[orders_placed] )
```

Orders Delivered

format `#,0`

```
SUM ( fact_zone_hour[orders_delivered] )
```

Fulfillment Rate %

format `0.0%`

Delivered orders as a share of placed orders in the current filter context.

```
DIVIDE ( [Orders Delivered], [Orders Placed] )
```

Cancellations

format `#,0`

```
[Orders Placed] - [Orders Delivered]
```

Cancellation Rate %

format `0.0%`

```
DIVIDE ( [Cancellations], [Orders Placed] )
```

No-Courier Cancels

format `#,0`

```
SUM ( fact_zone_hour[cancelled_no_courier] )
```

No-Courier Rate %

format 0.00%

Orders cancelled because no courier could be assigned — the sharpest liquidity signal.

```
DIVIDE ( [No-Courier Cancels], [Orders Placed] )
```

Customer Cancels

format #,0

```
SUM ( fact_zone_hour[cancelled_customer] )
```

ETA p50 (min)

format 0.0

Order-weighted mean of the per-zone-hour median ETA.

```
DIVIDE ( SUMX ( fact_zone_hour, fact_zone_hour[eta_p50_min] * fact_zone_hour[orders_delivered] ), [Orders Delivered] )
```

ETA p90 (min)

format 0.0

Order-weighted mean of the per-zone-hour 90th-percentile ETA — the customer-pain metric.

```
DIVIDE ( SUMX ( fact_zone_hour, fact_zone_hour[eta_p90_min] * fact_zone_hour[orders_delivered] ), [Orders Delivered] )
```

Assignment Latency p90 (s)

format 0

```
DIVIDE ( SUMX ( fact_zone_hour, fact_zone_hour[assignment_latency_p90_s] * fact_zone_hour[orders_placed] ), [Orders Placed] )
```

Peak Fulfillment Rate %

format 0.00%

Fulfillment restricted to the 18:00–21:59 dinner peak.

```
CALCULATE ( [Fulfillment Rate %], dim_time_block[is_peak_block] = TRUE ( ) )
```

Peak ETA p90 (min)

format 0.0

```
CALCULATE ( [ETA p90 (min)], dim_time_block[is_peak_block] = TRUE ( ) )
```

02 Liquidity

Available Courier-Hours

format #,0.0

```
SUM ( fact_zone_hour[available_courier_hours] )
```

Demanded Courier-Hours

format #,0.0

Orders placed × the period's average service time (pickup + drive + drop).

```
SUM ( fact_zone_hour[demanded_delivery_hours] )
```

Liquidity Ratio

format 0.00

Available ÷ demanded courier-hours. < 1 = supply-short.

```
DIVIDE ( [Available Courier-Hours], [Demanded Courier-Hours] )
```

Idle Courier-Hours

format #,0.0

```
SUM ( fact_zone_hour[idle_courier_hours] )
```

Idle Courier Ratio %

format 0.0%

Logged-in courier-hours not on a run — supply present but unproductive.

```
DIVIDE ( [Idle Courier-Hours], [Available Courier-Hours] )
```

Unmet Demand

format #,0

Placed – delivered, the volume a liquidity fix would recover.

```
SUM ( fact_zone_hour[unmet_demand] )
```

Active Couriers

format #,0

```
CALCULATE ( DISTINCTCOUNT ( fact_courier_shift_block[courier_key] ), fact_courier_shift_block[courier_key] > 0 )
```

Supply-Short Peak Zone-Hours %

format 0.0%

Share of peak zone-hours where available < demanded courier-hours.

```
VAR peak =  
    FILTER (  
        fact_zone_hour,  
        fact_zone_hour[is_peak_block] = TRUE () && fact_zone_hour[orders_placed] > 0  
    )  
RETURN  
DIVIDE (  
    COUNTROWS ( FILTER ( peak, fact_zone_hour[liquidity_ratio] < 1 ) ),  
    COUNTROWS ( peak )  
)
```

Deliveries per Active Hour

format 0.00

```
DIVIDE ( SUM ( fact_courier_shift_block[deliveries] ), DIVIDE ( SUM ( fact_courier_shift_block[active_min] ), 60 ) )
```

03 Courier Economics

Courier Earnings (BRL)

format #,0

```
SUM ( fact_courier_shift_block[earnings_brl] )
```

Active Hours

format #,0.0

```
DIVIDE ( SUM ( fact_courier_shift_block[active_min] ), 60 )
```

Logged-in Hours

format #,0.0

```
DIVIDE ( SUM ( fact_courier_shift_block[logged_in_min] ), 60 )
```

Earnings per Active Hour (BRL)

format #,0.00

```
DIVIDE ( [Courier Earnings (BRL)], [Active Hours] )
```

Utilisation %

format 0.0%

Active ÷ logged-in minutes.

```
DIVIDE ( SUM ( fact_courier_shift_block[active_min] ), SUM ( fact_courier_shift_block[logged_in_min] ) )
```

Payout per Delivery (BRL)

format #,0.00

```
DIVIDE ( [Courier Earnings (BRL)], SUM ( fact_courier_shift_block[deliveries] ) )
```

Incentive Spend (BRL)

format #,0

```
SUM ( fact_courier_shift_block[bonus_brl] )
```

Incentive Spend as % of Payout

format 0.0%

```
DIVIDE ( [Incentive Spend (BRL)], [Courier Earnings (BRL)] )
```

Contribution Margin per Delivered Order (BRL)

format #,0.00

SwiftBite take (22% of basket + fee) – courier base+tip payout – payment/support cost, per delivered order. Excludes the experiment bonus.

```
VAR delivered = [Orders Delivered]
VAR take =
  0.22 * CALCULATE ( SUM ( fact_order[basket_value_brl] ), fact_order[outcome] = "delivered" )
  + CALCULATE ( SUM ( fact_order[delivery_fee_brl] ), fact_order[outcome] = "delivered" )
VAR payout =
  SUM ( fact_delivery[courier_base_payout_brl] ) + SUM ( fact_delivery[tip_brl] )
VAR support = 1.1 * delivered
RETURN
DIVIDE ( take - payout - support, delivered )
```

04 Experiment

Treated Zone-Days

format #,0

```
CALCULATE ( COUNTROWS ( fact_experiment_zone_day ), fact_experiment_zone_day[arm] = "treatment" )
```

Control Zone-Days

format #,0

```
CALCULATE ( COUNTROWS ( fact_experiment_zone_day ), fact_experiment_zone_day[arm] = "control" )
```

Fulfillment (Treated) %

format 0.0%

Mean fulfillment rate across treated zone-days (peak block).

```
CALCULATE ( AVERAGE ( fact_experiment_zone_day[fulfillment_rate] ), fact_experiment_zone_day[arm] = "treatment" )
```

Fulfillment (Control) %

format 0.0%

```
CALCULATE ( AVERAGE ( fact_experiment_zone_day[fulfillment_rate] ), fact_experiment_zone_day[arm] = "control" )
```

Fulfillment Lift (pp)

format +0.00;-0.00;0.00

Primary metric: treated – control mean fulfillment, in percentage points.

```
( [Fulfillment (Treated) %] - [Fulfillment (Control) %] ) * 100
```

Fulfillment Lift SE (pp)

format 0.00

Two-sample normal SE on the zone-day means. The cluster-robust (by zone) SE is a Phase-10 artifact.

```
VAR nt = [Treated Zone-Days]
VAR nc = [Control Zone-Days]
VAR vt = CALCULATE ( VAR.S ( fact_experiment_zone_day[fulfillment_rate] ), fact_experiment_zone_day[arm] = "treatment" )
VAR vc = CALCULATE ( VAR.S ( fact_experiment_zone_day[fulfillment_rate] ), fact_experiment_zone_day[arm] = "control" )
RETURN
SQRT ( DIVIDE ( vt, nt ) + DIVIDE ( vc, nc ) ) * 100
```

Fulfillment Lift CI Low (pp)

format +0.00;-0.00;0.00

```
[Fulfillment Lift (pp)] - 1.96 * [Fulfillment Lift SE (pp)]
```

Fulfillment Lift CI High (pp)

format +0.00;-0.00;0.00

```
[Fulfillment Lift (pp)] + 1.96 * [Fulfillment Lift SE (pp)]
```

Fulfillment Lift Significant?

```
IF ( [Fulfillment Lift CI Low (pp)] > 0, "Yes (95%)", "No" )
```

ETA p90 (Treated) min

format 0.0

```
CALCULATE ( AVERAGE ( fact_experiment_zone_day[eta_p90_min] ), fact_experiment_zone_day[arm] = "treatment" )
```

ETA p90 (Control) min

format 0.0

```
CALCULATE ( AVERAGE ( fact_experiment_zone_day[eta_p90_min] ), fact_experiment_zone_day[arm] = "control" )
```

ETA p90 Lift (min)

format +0.0;-0.0;0.0

Co-primary evidence (lower is better).

```
[ETA p90 (Treated) min] - [ETA p90 (Control) min]
```

No-Courier Rate (Treated) %

format 0.00%

```
CALCULATE ( AVERAGE ( fact_experiment_zone_day[cancel_no_courier_rate] ), fact_experiment_zone_day[arm] = "treatment" )
```

No-Courier Rate (Control) %

format 0.00%

```
CALCULATE ( AVERAGE ( fact_experiment_zone_day[cancel_no_courier_rate] ), fact_experiment_zone_day[arm] = "control" )
```

No-Courier Rate Lift (pp)

format +0.00;-0.00;0.00

Co-primary evidence (lower is better).

```
( [No-Courier Rate (Treated) %] - [No-Courier Rate (Control) %] ) * 100
```

Incentive Spend - Experiment (BRL)

format #,0

```
SUM ( fact_experiment_zone_day[incentive_spend_brl] )
```

Incremental Delivered Orders

format #,0

Treated orders delivered above the control fulfillment rate.

```
VAR lift = [Fulfillment (Treated) %] - [Fulfillment (Control) %]  
VAR treated_placed =  
    CALCULATE ( SUM ( fact_experiment_zone_day[orders_placed] ), fact_experiment_zone_day[arm] = "treatment" )  
RETURN  
lift * treated_placed
```

G1 Incentive Cost per Incremental Order (BRL)

format #,0.00

The go/no-go economics: bonus spend ÷ incremental delivered orders.

```
DIVIDE ( [Incentive Spend - Experiment (BRL)], [Incremental Delivered Orders] )
```

G1 Verdict

```
IF (  
    [G1 Incentive Cost per Incremental Order (BRL)] < [Contribution Margin per Delivered Order (BRL)],  
    "Pays for itself",  
    "Costs more than it saves"  
)
```

Fulfillment Lift - Short tier (pp)

format +0.00;-0.00;0.00

```
VAR t = CALCULATE ( AVERAGE ( fact_experiment_zone_day[fulfillment_rate] ),
    fact_experiment_zone_day[arm] = "treatment", dim_zone[baseline_supply_stress_tier] = "short" )
VAR c = CALCULATE ( AVERAGE ( fact_experiment_zone_day[fulfillment_rate] ),
    fact_experiment_zone_day[arm] = "control", dim_zone[baseline_supply_stress_tier] = "short" )
RETURN ( t - c ) * 100
```

Fulfillment Lift - Balanced tier (pp)

format +0.00;-0.00;0.00

```
VAR t = CALCULATE ( AVERAGE ( fact_experiment_zone_day[fulfillment_rate] ),
    fact_experiment_zone_day[arm] = "treatment", dim_zone[baseline_supply_stress_tier] = "balanced" )
VAR c = CALCULATE ( AVERAGE ( fact_experiment_zone_day[fulfillment_rate] ),
    fact_experiment_zone_day[arm] = "control", dim_zone[baseline_supply_stress_tier] = "balanced" )
RETURN ( t - c ) * 100
```

Fulfillment Lift - Long tier (pp)

format +0.00;-0.00;0.00

```
VAR t = CALCULATE ( AVERAGE ( fact_experiment_zone_day[fulfillment_rate] ),
    fact_experiment_zone_day[arm] = "treatment", dim_zone[baseline_supply_stress_tier] = "long" )
VAR c = CALCULATE ( AVERAGE ( fact_experiment_zone_day[fulfillment_rate] ),
    fact_experiment_zone_day[arm] = "control", dim_zone[baseline_supply_stress_tier] = "long" )
RETURN ( t - c ) * 100
```

Arm x Tier Contrast (Short minus Long, pp)

format +0.00;-0.00;0.00

Descriptive heterogeneity contrast. The formal LR interaction test is Phase 10.

[Fulfillment Lift - Short tier (pp)] - [Fulfillment Lift - Long tier (pp)]

Cannibalisation - Adjacent-Control Gap (pp)

format +0.00;-0.00;0.00

Fulfillment on control zone-days next to a treated zone minus control zone-days that are not. Negative = neighbours are being drained (cannibalisation).

```
VAR adj =
    CALCULATE ( AVERAGE ( fact_experiment_zone_day[fulfillment_rate] ),
        fact_experiment_zone_day[arm] = "control", fact_experiment_zone_day[is_adjacent_to_treated] = TRUE ( ) )
VAR noadj =
    CALCULATE ( AVERAGE ( fact_experiment_zone_day[fulfillment_rate] ),
        fact_experiment_zone_day[arm] = "control", fact_experiment_zone_day[is_adjacent_to_treated] = FALSE ( ) )
RETURN ( adj - noadj ) * 100
```

Cannibalisation-Adjusted Net Lift (pp)

format +0.00;-0.00;0.00

Rough metro-level net of the local lift and the neighbour-zone drag. Phase 10 does the full spatial model.

[Fulfillment Lift (pp)] + [Cannibalisation - Adjacent-Control Gap (pp)]

Balance - Pre-period Fulfillment Gap (pp)

format +0.00;-0.00;0.00

Should be ≈ 0 — a smoke test that randomisation balanced the arms on the 15-month baseline.

```
VAR t = CALCULATE ( AVERAGE ( fact_incentive_assignment[pre_fulfillment_rate] ), fact_incentive_assignment[arm] = "treatment" )
VAR c = CALCULATE ( AVERAGE ( fact_incentive_assignment[pre_fulfillment_rate] ), fact_incentive_assignment[arm] = "control" )
RETURN ( t - c ) * 100
```

Settled Lift (weeks 8-10, pp)

format +0.00;-0.00;0.00

Lift over the last three weeks — after any launch-novelty bump has decayed.

```
VAR t = CALCULATE ( AVERAGE ( fact_experiment_zone_block_week[fulfillment_rate] ),  
                    fact_experiment_zone_block_week[arm] = "treatment", fact_experiment_zone_block_week[experiment_week] >= 8 )  
VAR c = CALCULATE ( AVERAGE ( fact_experiment_zone_block_week[fulfillment_rate] ),  
                    fact_experiment_zone_block_week[arm] = "control", fact_experiment_zone_block_week[experiment_week] >= 8 )  
RETURN ( t - c ) * 100
```

Weekly Fulfillment %

format 0.0%

Mean zone-block fulfillment per experiment week.

```
AVERAGE ( fact_experiment_zone_block_week[fulfillment_rate] )
```

Weekly Fulfillment (Treated) %

format 0.0%

```
CALCULATE ( AVERAGE ( fact_experiment_zone_block_week[fulfillment_rate] ), fact_experiment_zone_block_week[arm] = "treatment" )
```

Weekly Fulfillment (Control) %

format 0.0%

```
CALCULATE ( AVERAGE ( fact_experiment_zone_block_week[fulfillment_rate] ), fact_experiment_zone_block_week[arm] = "control" )
```

Zone-Day Fulfillment %

format 0.0%

Mean fulfillment across the zone-days in context — split by `arm` for the guardrail table.

```
AVERAGE ( fact_experiment_zone_day[fulfillment_rate] )
```

Zone-Day ETA p90 (min)

format 0.0

```
AVERAGE ( fact_experiment_zone_day[eta_p90_min] )
```

Zone-Day No-Courier Rate %

format 0.00%

```
AVERAGE ( fact_experiment_zone_day[cancel_no_courier_rate] )
```

Zone-Day Earnings per Active Hour (BRL)

format #,0.00

```
AVERAGE ( fact_experiment_zone_day[courier_earnings_per_active_hour_brl] )
```

Zone-Day Incentive Cost per Order (BRL)

format #,0.00

```
AVERAGE ( fact_experiment_zone_day[incentive_cost_per_delivered_order_brl] )
```

05 Targets & Context

Fulfillment vs Target (pp)

format +0.0;-0.0;0.0

Plan: ≥ 98% metro-wide (doc 01 §10).

$$([\text{Fulfillment Rate \%}] - 0.98) * 100$$

Peak ETA p90 vs Target (min)

format +0.0;-0.0;0.0

Plan: ≤ 55 min at peak. Lower is better — good when negative.

$$[\text{Peak ETA p90 (min)}] - 55$$

No-Courier Rate vs Target (pp)

format +0.00;-0.00;0.00

Plan: ≤ 0.5%. Lower is better.

$$([\text{No-Courier Rate \%}] - 0.005) * 100$$

Idle Courier Ratio vs Target (pp)

format +0.0;-0.0;0.0

Plan: ≤ 25%. Lower is better.

$$([\text{Idle Courier Ratio \%}] - 0.25) * 100$$

06 Exec Insight

Exec Insight

One-line dynamic read of the experiment for the Executive Overview page.

```
VAR lift = [Fulfillment Lift (pp)]
VAR sig = [Fulfillment Lift CI Low (pp)] > 0
VAR g1 = [G1 Incentive Cost per Incremental Order (BRL)]
VAR cm = [Contribution Margin per Delivered Order (BRL)]
VAR eta = [ETA p90 Lift (min)]
VAR cann = [Cannibalisation - Adjacent-Control Gap (pp)]
RETURN
"Zone-hour incentive: fulfillment " &
  IF ( lift >= 0, "+", "" ) & FORMAT ( lift, "0.0" ) & " pp" &
  IF ( sig, " (significant at 95%)", " (not significant)" ) &
  ", ETA p90 " & FORMAT ( eta, "+0.0;-0.0" ) & " min. " &
  "Cost per incremental order R$ " & FORMAT ( g1, "0.00" ) &
  IF ( g1 < cm, " – below", " – above" ) & " the R$ " & FORMAT ( cm, "0.00" ) &
  " contribution margin. Neighbour zones " &
  IF ( cann < -0.3, "show a drag of " & FORMAT ( cann, "0.0" ) & " pp (cannibalisation).",
    "show no material drag." )
```

SwiftBite Delivery — Visual Identity

Document status: Phase 5 deliverable. The single source of truth is `assets/brand.py`, imported by `assets/gen_logo.py`, the Phase 6/8 Power BI scripts and the Phase 10 board deliverables so the logo, page backgrounds, `theme.json` and every scripted override draw from the same values.

PT. Identidade própria do SwiftBite — não reutiliza navy do AeroVanti/VoltEdge nem violeta do Voxa+. Uma cor de marca (verde-floresta), um accent (tangerina — zona tratada / período atual / "olhe aqui"), uma cor de comparação (cinza-pedra frio — braço de controle / período anterior / meta), neutros mornos e gridlines discretas.

Tone

Corporate, restrained, operations- and finance-facing. Delivery-adjacent, not a marketing page — this is a marketplace-liquidity diagnosis and a randomized incentive-experiment read-out.

Palette

Role	Hex	Use
SwiftBite green	#1E6B4F	brand, primary series, table accent, hyperlink
Green-deep	#113A2C	header band, darkest ink on light
Tangerine (accent)	#E8823C	treated zone · current period · "look here"
Stone (comparison)	#8090A0	control arm · prior period · target · secondary series
Page ground	#F2F4F1	warm-neutral, faintly green off-white
Card	FFFFFF	
Gridline / hairline	#E6E9E4 / #D6DAD3	quiet
Good / Bad / Neutral	#2E7D5B / #C0453B / #8A8A99	semantic; ETA, cancellation rate, idle-courier ratio and incentive cost are lower-is-better

Categorical ramp: green, stone, tangerine, teal `#3E9C93`, plum `#8E5EA8`, ochre `#C9922E`, rose `#C77CA6`, grey `#6B7280`.

Supply-stress tier ramp (supply-short zones read "hot"): short `#E8823C` · balanced `#8090A0` · long `#1E6B4F`. **Experiment arm:** treatment `#E8823C` · control `#8090A0`.

Diverging scale (vs-target, lift, spillover): `#C0453B` → `#EAD9C4` → `#2E7D5B`.

Type

Segoe UI family. Semibold for titles / callouts, Regular for body / labels. Callout (KPI value) 30 px, title 15 px, header 12 px, label 9 px. Inks: title `#17241E`, body `#33403A`, muted `#6B7A72`; on dark `#F0F5F1` / `#B7C7BD`.

Signature element (every page)

Green-deep header band (58 px) with the **SwiftBite** wordmark (tangerine "Bite"), a tangerine 3 px hairline under it, a right-aligned "Marketplace Analytics" label, then the page title + its one business question on the light body, and a hairline divider. Below: a slicer row (Analysis period · Arm · Zone / stress tier), a white rounded KPI frame with 6 value cards, then two content rows on the 3-30-300 gradient.

Assets emitted (`python assets/gen_logo.py` → `powerbi/assets/`)

- `logo_mark_on_light.png` / `_on_dark.png` — rounded-square tile, two forward speed chevrons ("swift") + a trailing tangerine dot ("bite").
- `wordmark_on_light.png` / `_on_dark.png` — "SwiftBite" lockup, tangerine "Bite".
- `bg_01_executive.png` (1280×860) · `bg_02_health.png` · `bg_03_experiment.png` · `bg_04_courier.png` (1280×720) — full-page backgrounds carrying the band + title.
- `_page_heights.json` — page → canvas height, read in Phase 8.

Theme

`powerbi/theme/theme.json` — `dataColors` (the 8-colour ramp), semantic colours, the diverging min/center/max, 4 text classes, and a `visualStyles` baseline: white card background, 8 px rounded `#D6DAD3` border, no drop shadow, no visual header, and the page ground `#F2F4F1`. The per-visual-type overrides (data labels, axis gridlines, KPI/card specifics) are promoted into the theme during Phase 6/8 via the `pbir` CLI; `pbir theme validate` must pass.

End of Phase 5 deliverable.

SwiftBite Delivery — Data Quality

Document status: Phase 4 deliverable. **Depends on:** `02_data_architecture.md`, `04_etl_pipeline.md`. **Companion artifact:** `data/quality/dq_report.md` (regenerated by `python etl/dq_checks.py`; the one data file committed to the repo).

PT. Como a sujeira é injetada de propósito, onde é tratada, e o que o build recusa a deixar passar. Oito classes (Q01–Q08), cada uma com um contador no `dq_report.md` e, quando é um invariante, uma asserção dura em `dq_checks.py`.

1. Philosophy

Every messy trait is **injected on purpose** by `01_generate_raw.py`, **handled** by `02` (parse / repair) or `03` (conform / sentinel), **counted** into a per-entity DQ fragment, and — where it is an invariant — **asserted** in `dq_checks.py`, which fails the build. Raw is never edited after generation. One `SEED` makes the whole thing reproducible, so the counters below are stable.

2. The eight injected classes

#	Issue	Injected in	Handled in	Hard check
Q01	Mixed datetime formats — DD/MM/YYYY HH:MM vs ISO in OrderCore; Unix-epoch ms in the GPS reference; Excel serials for PayHub settlement weeks	ordercore, trace, payhub	<code>02</code> split parser (<code>^\d{4}-</code> → ISO fast path, else BR); <code>from_excel_serial</code>	<code>datetimes_unparseable == 0</code>
Q02	Timezone drift — PayHub & Boost export UTC <code>Z</code> with no offset; OrderCore & RiderApp are Brazil-local with no marker	payhub, boost vs ordercore, riderapp	<code>02</code> localises the naive feeds <code>America/Sao_Paulo</code> → UTC, parses the <code>Z</code> feeds as UTC; <code>03</code> derives the local calendar day/hour for keys	no naive timestamps in curated; peak-block bucketing exact
Q03	Money as locale strings — "R\$ 62,90", "1.234,56", "(12,50)" (reversal)	ordercore basket, payhub amounts	<code>02</code> <code>parse_money</code> (strip <code>R\$</code> /thousands, <code>,</code> → <code>.</code> , parens → negative)	no non-null money column is object dtype; payout / basket ≥ 0
Q04	GPS noise — lat/long jitter, far-away cold-start pings, <code>(0,0)</code> null-island, irregular cadence	trace <code>location_pings</code> (7-day reference)	<code>02</code> bbox + null-island drop, resample to courier-minute state	0 out-of-bbox pings survive into <code>stg_trace_ping_minute</code>
Q05	Duplicate rows — mobile retry dupes on orders; at-least-once status / offer / bonus logs; a re-delivered PayHub month	ordercore, riderapp, dispatch, payhub	<code>02</code> exact-dedup; drop the re-delivered file on <code>order_id</code>	<code>fact_order</code> / <code>fact_delivery</code> key-unique; <code>fact_experiment_zone_day</code> unique on (<code>zone_key</code> , <code>date_key</code>)
Q06	Orphan keys — a slice of <code>offers</code> reference a <code>courier_id</code> absent from the master	riderapp	<code>03</code> sentinel <code>courier_key = -2</code> , counted	every fact FK resolves to a real or sentinel key
Q07	Zone boundary redraw (window-month 9) — the as-booked label is stale for the two re-cut zones	georef + ordercore label	<code>03</code> keeps both <code>dropoff_zone_key_asbooked</code> and <code>_current</code> (centroid-resolved only for the redrawn pair pre-redraw); experiment/zone-hour facts key off <code>_current</code>	every order has a non-null <code>dropoff_zone_key_asbooked</code> ; drift reported
Q08	Encoding damage — Latin-1 mojibake in address <code>bairro</code> / <code>logradouro</code> and restaurant names	ordercore	<code>02</code> <code>ftfy.fix_text</code> on the text columns	no <code>Ã</code> / <code>Â</code> mojibake sequences remain in curated address / restaurant text

Also handled without ceremony: `courier_id` casing → upper-normalised; post-midnight Boost campaign edits → earliest (`00:00` -fixed) row per zone-day; the `no_courier_found` → late-assignment funnel → one reconciled outcome per order; `status` / offer `outcome` / cuisine free-text → controlled vocab in `etl/vocab/`.

3. Operational identities asserted (hard)

- Order conservation**, every zone-hour: `orders_placed == orders_delivered + Σ cancelled_{customer, courier, no_courier, restaurant}` (exact).
- Rate sanity**: `fulfillment_rate ∈ [0, 1]`; `eta_p50_min ≤ eta_p90_min`; `liquidity_ratio ≥ 0`; `idle_courier_ratio ∈ [0, 1]`.

- **Experiment integrity:** every `fact_experiment_zone_day` zone-day exists in `fact_incentive_assignment` ; treated zone-days have `bonus_br1_per_delivery > 0` , control `== 0` ; ≥ 75 days of post-experiment fact data.

4. Benchmark bands checked (soft — reported, not fatal)

On the **pre-experiment baseline** (window-months 1–15) the following must sit mid-band, or the calibration is off (see `01_business_context_kpis.md` §9):

Metric	Band	Current build
Fulfillment rate	0.93 – 0.995	0.956 ✓
ETA p50 (min)	24 – 42	≈ 38 ✓
ETA p90 (min)	42 – 80	≈ 46 ✓
Cancellation rate	0.01 – 0.07	0.044 ✓
Idle-courier ratio	0.10 – 0.40	0.286 ✓
Courier earnings / active-hour (BRL)	20 – 40	≈ 33 ✓
Per-stratum arm allocation	0.45 – 0.55	0.50 all 12 ✓

The experiment arms are allowed to diverge inside the window — that is the point.

5. Current status

`python etl/dq_checks.py` → **13 / 13 hard checks pass · 0 soft warnings**. The full counter table (rows in / rows out, dupes dropped, money strings parsed, GPS pings dropped, mojibake repaired, late edits reconciled, ...) is in [data/quality/dq_report.md](#) .

End of Phase 4 deliverable (Data Quality).

SwiftBite Delivery — Report Structure

Document status: Phase 8 deliverable. The report is built from code: `powerbi/gen_report_visuals.py` writes `powerbi/_report_layout.json`, `powerbi/_drive_pbir.py` drives the `pbir` CLI to produce `powerbi/SwiftBiteDelivery.Report/**`. `powerbi/build_report.sh` orchestrates the whole chain. `pbir validate --all` passes.

PT. 4 páginas, uma pergunta de negócio por página. Cada página: banda verde com o wordmark, hairline tangerina, título + a pergunta, uma linha de 6 KPI cards numa moldura branca, e 4 visuais de conteúdo. 3 slicers globais (período / zona / tier de estresse de oferta).

Every page

- **Header band** (green-deep, 58 px) with the *SwiftBite* wordmark, a tangerine hairline, and a right-aligned "Marketplace Analytics" label — carried by the page-background PNG (`assets/gen_logo.py`).
- **Title + business question** on the light body.
- **Slicer row:** Analysis period (`dim_date[date]`) · Zone (`dim_zone[zone_name]`) · Supply-stress tier (`dim_zone[baseline_supply_stress_tier]`).
- **KPI frame:** a white rounded shape holding **6 value cards**.
- **Two content rows** on the 3-30-300 gradient (4 visuals).

All numbers come from the 71 explicit DAX measures in `_Measures` (`discourageImplicitMeasures` is on); see `06_dax_measures.md` .

1 · Executive Overview

Question — Is the marketplace healthy, and did the zone-hour incentive earn its cost?

KPIs	Visuals
Fulfillment Rate · Peak ETA p90 · No-Courier Rate · Idle Courier Ratio · Incentive Lift (pp) · Cost / Incremental Order	fulfillment-rate trend (line) · Exec Insight dynamic narrative card · fulfillment lift by supply-stress tier (bar) · weekly fulfillment treated vs control (line)

2 · Marketplace Health

Question — Where and when does liquidity break?

KPIs	Visuals
Orders Placed · Fulfillment Rate · ETA p90 · No-Courier Rate · Liquidity Ratio · Idle Courier Ratio	fulfillment rate by zone (bar) · liquidity ratio by zone × daypart (matrix) · ETA p90 by hour of day (line) · cancellations by cause and zone (column)

3 · Pricing & Incentive Experiment

Question — Is the incentive real, safe on the guardrails, and not just cannibalising neighbour zones?

KPIs	Visuals
Fulfillment Lift (pp) · ETA p90 Lift (min) · No-Courier Lift (pp) · Cost / Incremental Order · Neighbour-Zone Gap (pp) · Adjusted Net Lift (pp)	fulfillment lift by tier — heterogeneity (bar) · fulfillment by arm (column) · weekly fulfillment treated vs control — launch bump vs settled (line) · guardrails by arm (table)

4 · Courier Economics

Question — What do couriers earn, how utilised is supply, and how elastic is it to the bonus?

KPIs	Visuals
Active Couriers · Earnings / Active Hr · Utilisation · Deliveries / Active Hr · Contribution Margin · Incentive % of Payout	earnings per active hour by daypart (column) · utilisation by zone (bar) · available vs demanded courier-hours by hour (line) · incentive cost per delivered order by tier (column)

Layout warnings from `pbir validate` (`VISUAL_OVERLAP` / `VISUAL_UNDSIZED`) are expected — the KPI value cards sit inside the frame shape by design; final pixel spacing is a Power BI Desktop polish pass.

End of Phase 8 deliverable (Report Structure).

SwiftBite Delivery — build walkthrough

A log of how this project was built, phase by phase — decisions, dead ends, and calibration passes.

Phase 0 — scope & scaffold

Locked the brief: one Brazilian metro (Belo Horizonte), **12 zones** with an adjacency graph, BRL only, **20-month** window (2025-01 → 2026-08), a **10-week** zone-hour incentive experiment in months 16–18, **randomization unit = zone-day**. Chose a supply-side intervention on a two-sided marketplace specifically because it is a rarer, higher-value problem shape than another conversion A/B test, and because it carries a real **neighbour-zone cannibalisation** risk (spatial confounding).

Folder scaffold mirrors the other three portfolio projects (`docs/00-09` , `etl/` , `powerbi/` , `data_analysis/` , `portfolio/`).

Phase 1 — business context & KPIs

`docs/01_business_context_kpis.md` : the two marketplace sides kept strictly separate (supply = available courier-hours, demand = demanded delivery-hours; liquidity = their ratio per zone × hour). Primary metric = **fulfillment rate**; five guardrails including **G1 incentive cost per incremental delivered order** (the go/no-go economics) and **G5 neighbour-zone spillover**. Power/MDE worked for the **zone-day** unit (~420/arm, ICC ≈ 0.15, design effect ≈ 1.9 → pooled MDE ≈ +1.3 pp, short-tier ≈ +2.2 pp), with cluster-robust SEs + randomization inference as the small-cluster cross-checks.

Phase 2 — data architecture

`docs/02_data_architecture.md` : medallion, one `SEED` . Six simulated systems (OrderCore, RiderApp, Trace GPS, Dispatch telemetry, PayHub, Boost) + a static GeoRef. Curated star: `dim_zone` (+ `dim_zone_adjacency` bridge), `dim_time_block` , `fact_zone_hour` as the liquidity spine, `fact_incentive_assignment` + `fact_experiment_zone_day` at the zone-day grain. Eight data-quality classes (Q01–Q08) with hard checks. The true effect and the cannibalisation term stay only in `etl/config.py` .

Phase 3 — ETL pipeline

`etl/` — `config.py` , `utils.py` , `01_generate_raw.py` , `02_clean_stage.py` , `03_build_curated.py` , `dq_checks.py` , `run_pipeline.py` , `vocab/` .

01 is a marketplace simulator, vectorised at the zone-hour grain: per zone × day × hour it draws demand (Poisson around a shape with day-of-week, month, rainy-season, per-day rain and local-event multipliers), allocates courier supply by a demand-share × attractiveness tilt (so central zones run long and peripheral zones run short), computes the **liquidity ratio**, and clears the market through smooth `fulfillment / ETA / cancellation` curves in `L` . On a treated peak zone-day it multiplies effective supply by a tier-dependent response (short 1.34× / balanced 1.15× / long 1.04×) with a week-1 novelty bump that decays, and **pulls a 28% share of that added supply out of adjacent control zones** (cannibalisation). It then emits messy source exports (mixed datetime formats, epoch-ms GPS, UTC-vs-local feeds, money-as-text, retry duplicates, orphan courier ids, the zone boundary redraw, mojibake).

02 parses/types/dedups each raw entity → `stg_*` , reduces the 7-day GPS ping reference to courier-minute state, reconciles the assignment funnel to one outcome per order, and reconciles the Boost post-midnight campaign edits to the 00:00 assignment. **03** builds the Kimball star: resolves each order's zone two ways (as-booked label vs current, recovered from the address centroid), puts supply and demand in courier-hours, builds `fact_zone_hour` , derives the baseline supply-stress tier from months 1–15 liquidity, and pre-computes the zone-day experiment tables.

Calibration passes (Phase 3)

- Supply allocation.** First cut put 10/12 zones "short" with liquidity from 0.27 to 3.7 — the softmax zone-choice model concentrated supply into 2–3 zones. Fix: allocate supply proportional to *courier-hour* demand share (not order share — service time differs by zone) times a small attractiveness tilt $\exp(0.20 \cdot z)$. Landed at 6 short / 2 balanced / 4 long, baseline liquidity 0.77–1.20.
- OneDrive file-locks.** The project lives in a synced folder; `rm -rf data/` + rewrite fought OneDrive and the atomic-write retries stalled the run. Fix: `SB_DATA_DIR` env var points the regenerated layers outside the synced tree; `_atomic_write` falls back to an in-place write after retries.
- Runtime.** First **01** took 3.5 min (per-order Python loop). Vectorised order-attribute generation per zone-hour → ~70 s. **02** datetime parsing with `format="mixed"` was ~3.5 min → split ISO/BR fast paths → ~35 s. **03** courier→delivery assignment per-row loop → per-group choice call. Full pipeline now ~6.5 min (**03** ~4.5 min).
- Marketplace bands.** Raised metro supply ratio 1.05 → 1.12 and softened the `fulfillment(L)` / cancellation-split curves (no-courier is a minority of failures, not the majority). Baseline now: fulfillment 0.956, ETA p50 38 / p90 46 min, cancel 4.4 %, no-courier 1.4 %, idle 29 %, courier earnings R\$33/active-hr — all mid-band.

Result: `dq_checks` — **13/13 hard checks pass, 0 soft warnings**. Naive experiment read (not the Phase-10 analysis): fulfillment control 0.957 → treatment 0.975 (+**1.7 pp**), ETA p90 47 → 41 min, no-courier 1.2 % → 0.8 %; heterogeneity short (+2.3 pp) > balanced (+1.9) > long (+0.7).

Decision taken: keep the calibration and let Phase 10 carry **ETA p90 and no-courier rate as co-primary evidence** alongside fulfillment (option B) — a maturity-realistic framing (a #2 player already fulfils ~96 %; the customer-felt pain is ETA and cancels).

Phase 4 — data documentation

- `etl/gen_data_dictionary.py` introspects `data/curated/*.parquet` → `docs/03_data_dictionary.md` (15 tables, column / dtype / non-null % / distinct / example, + a hand-written blurb each).
- `docs/04_etl_pipeline.md` — what each script guarantees and how to run it.
- `docs/05_data_model.md` — bus matrix, grain & measures per fact, dimension attributes, the full relationship table (three zone roles on `fact_order` ; only dropoff-current active), an ER diagram, and the Phase-7 measure-group preview.
- `docs/08_data_quality.md` — the eight injected classes Q01–Q08, the hard operational identities, the benchmark bands, and the current status.

Two DQ gaps found and fixed during this phase: 1. **Q08 was a no-op** — the generated address / restaurant text was pure ASCII, so `mojibake()` had nothing to corrupt. Gave the generator accented BH street / bairro names ("Funcionários", "São Pedro", "Rua Antônio de Albuquerque", ...); now `mojibake_repaired` = ~69k address fields + 46 restaurants. 2. **Inflated zone drift** — recovering `dropoff_zone_key_current` by nearest-centroid on jittered address points mislabelled ~13 % of orders. Fixed: `_current` = the as-booked label everywhere **except** the two redrawn zones on pre-redraw dates (the only place the label is legitimately stale). Drift dropped 47k → 4.4k (~1.2 %, the redraw pair only), and orphan-courier deliveries went to 0 with a zone-day courier fallback.

Pipeline after the fixes: **13/13 hard pass · 0 soft warnings**, ~8 min end-to-end (`03` ~6.5 min — the slow step; `run_pipeline.py --from 03` reuses raw + staging).

Phase 5 — visual identity

Palette chosen from three options: **forest green** `#1E6B4F` + **tangerine** `#E8823C` **accent** + **cool stone** `#8090A0` **comparison**, warm-neutral ground `#F2F4F1`. Deliberately clear of the siblings (AeroVanti sky-blue/navy, VoltEdge navy, Voxa+ violet). Supply-short zones and the treated arm both map to tangerine ("hot"); healthy / long zones and the control arm map to green / stone.

- `assets/brand.py` — the single source of truth (palette, tier & arm ramps, type, layout constants, the 4 page titles + questions).
- `assets/gen_logo.py` → `powerbi/assets/` — mark (two forward speed chevrons + a trailing tangerine dot), "SwiftBite" wordmark, four full-page backgrounds carrying the green header band + tangerine hairline + title, `_page_heights.json`.
- `powerbi/theme/theme.json` — data-colour ramp, semantic + diverging colours, 4 text classes, a `visualStyles` baseline (white card, 8 px `#D6DAD3` border, no shadow, page ground). Per-visual-type overrides come in Phase 6/8 via the `pbir` CLI.
- `docs/07_visual_identity.md`.

Phase 6 — semantic model as code (TMDL)

`powerbi/gen_semantic_model.py` reads the curated Parquet schemas and writes `powerbi/SwiftBiteDelivery.SemanticModel/**` as TMDL: **15 tables** (import partitions via a `pDataFolder` parameter, committed with a `C:/PATH/TO/REPO/...` placeholder) + an empty `_Measures` table that Phase 7 fills.

- All facts are day/hour grain, so every fact joins `dim_date[date_key]` (unique — no `dim_month` helper needed, unlike AeroVanti) and, where relevant, `dim_time_block[time_block_key]`.
- **fact_order** **role-plays** **dim_zone** **three ways**: dropoff-current (active), dropoff-as-booked and pickup (both inactive, for `USERRELATIONSHIP` measures).
- `dim_zone_adjacency` is a bridge (`dim_zone[zone_key] 1:* dim_zone_adjacency[zone_key]`); `neighbour_zone_key` is measure-driven, not a physical relationship.
- Sentinel courier rows `-1` (no courier) / `-2` (orphan) added to `dim_courier` so every `fact_order` / `fact_delivery` FK resolves.
- `summarizeBy` set from the column name: `sum` for additive counts / amounts / hours, `none` for keys, rates, ratios, `*_p50/p90`, `per_active_hour`, `pre_*` covariates.
- **28 relationships** (2 inactive). `fact_delivery` gained `date_key` / `time_block_key` in `03` so it joins the calendar directly rather than through `fact_order`.

Full `pbir` validation runs in Phase 8 alongside the report.

Phase 7 — DAX measure library

`etl/pbi_measures.py` holds **63 measures** in 6 display folders; `powerbi/gen_semantic_model.py` injects them into `_Measures.tmdl` and `etl/gen_dax_doc.py` renders `docs/06_dax_measures.md` from the same list.

- **01 Marketplace Health** — orders, fulfillment rate, cancellation & no-courier rate, order-weighted ETA p50 / p90, assignment latency, peak-only variants.
- **02 Liquidity** — available vs demanded courier-hours, liquidity ratio, idle-courier ratio, unmet demand, supply-short peak zone-hours %, deliveries/active-hour.
- **03 Courier Economics** — earnings, active hours, utilisation, payout/delivery, incentive spend, and **contribution margin per delivered order** (take – payout – support).
- **04 Experiment** — treated/control fulfillment, **lift (pp) + two-sample normal CI**, ETA p90 lift, no-courier lift, incremental delivered orders, **G1 incentive cost per incremental order + verdict vs contribution margin**, per-tier lift (short/balanced/long)
- short–long contrast, **cannibalisation: adjacent-control gap + cannibalisation-adjusted net lift**, pre-period balance gap, settled lift (weeks 8–10).
- **05 Targets & Context** — vs-target deltas (fulfillment ≥ 98%, peak ETA p90 ≤ 55, no-courier ≤ 0.5%, idle ≤ 25%).
- **06 Exec Insight** — one dynamic narrative measure for the Executive Overview.

The live experiment CI is deliberately the simple two-sample interval; the cluster-robust SE, randomization-inference p and the formal arm×tier interaction test are Phase-10 work, not measures. Measure names kept ASCII (no em-dash / ×) for `pbir` safety.

Phase 8 — report build

`powerbi/gen_report_visuals.py` writes `_report_layout.json` (4 pages, one business question each: Executive Overview · Marketplace Health · Pricing & Incentive Experiment · Courier Economics); `powerbi/_drive_pbir.py` turns that manifest into `SwiftBiteDelivery.Report/**` via the `pbir` CLI (`new report` → `rebind --local` → `add page / background` → `add visual --from-json` per page → `sort / conditional-format / KPI-frame` styling pass → `validate`). `powerbi/build_report.sh` is the full orchestrator (regenerates brand assets + model + DAX doc + layout first; aborts if Power BI Desktop is running so `pbir` validates the TMDL, not a stale live model).

Each page: a KPI frame with 6 value cards, 4 content visuals, and 3 slicers (period / zone / supply-stress tier). 56 visuals total.

Two build fixes: 1. `pbir lineChart` rejects a `Series` role — the two "weekly fulfillment by arm" line charts were rewritten as two explicit Y measures (Weekly Fulfillment (Treated) % / (Control) %), no series. 2. OneDrive held a lock on the just-deleted `.Report` tree, so `pbir new report` half-created it. `scaffold()` now retries the delete up to 6× with a back-off.

`pbir validate --all` → "✓ **SwiftBiteDelivery validation passed**" (4 pages, 56 visuals, 116 fields, model loaded). The 93 advisory warnings are `VISUAL_OVERLAP` / `VISUAL_UNDSIZED` — the KPI cards sit inside the frame shape by design; final spacing is a Power BI Desktop polish pass (as with the sibling projects).

Phase 9 — build log

This document. Maintained per phase, not written at the end.

Phase 10 — the read-out (primary deliverable)

`data_analysis/` — dual-track (pandas + DuckDB) analysis of the zone-hour incentive experiment. `utils.py` carries the loaders and the three inference routes the zone-day design needs: cluster-robust (by zone) OLS SEs, **randomization inference** (permute `arm` within zone × weekday/weekend, 3,000 draws), and a **wild-cluster bootstrap** (Rademacher, resampled by zone). Eight `python/NN_*.py` scripts each compute their headline number on both tracks and call `utils.parity(...)`; `parity/run_all.py` runs them all — **8/8 scripts, every parity check passes**. `sql/NN_*.sql` is the readable SQL reference.

#	Analysis	Result
01	anchor	840 zone-days, 420/arm; naive fulfillment lift +1.87 pp
02	balance	exact 50/50 by design; zone-level covariates balance by construction (SMD = 0)
03	primary	fulfillment +1.87 pp (cluster CI [+1.27, +2.46]; RI p = 3×10 ⁻⁴ ; wild-boot [+1.20, +2.55]); ETA p90 −6.06 min; no-courier −0.36 pp
04	guardrails	ops guardrails all favourable; G1 fails — R\$ 281 per incremental delivered order vs R\$ 10 margin
05	novelty	no detectable decay trend (weekly series too thin)
06	heterogeneity	short +2.29 · balanced +2.43 · long +0.94 pp ; interaction Wald p = 0.004; short-CI ∩ long-CI = ∅
07	cannibalisation	≈ 17% of local, not significant (adjacent-control gap p = 0.76); metro net ≈ 83%
08	synthesis	don't ship the flat bonus; restructure to reward incrementality; cap to short + balanced zones

Two calibration calls made while writing the read-out: (1) G1's cost-per-incremental-order is computed as a **pooled** Σ/Σ ratio with a zone-cluster bootstrap — a per-zone ratio with clipping had inflated it via near-zero denominators in ~zero-effect zones. (2) The result is deliberately a **"do not roll out"** with a restructure path — the intervention improves the marketplace but is priced wrong (flat bonus on every delivery, ~2% incremental). This is a stronger analytical demonstration than a "yes, ship it" and matches the ceiling- compressed primary metric flagged back in Phase 3.

`findings/01–08` write each analysis up; `deliverables/build_deliverables.py` renders the decision-first **board summary** and the exhaustive **deep dive** to HTML + PDF in the SwiftBite identity, embedding the Love / novelty / forest figures.

Phase 11 — portfolio assets

`portfolio/gen_portfolio_images.py` → **5 gallery images only** (1600×820, SwiftBite green): a branded cover with the headline result, the medallion pipeline, the curated star schema, six key DAX measures as code cards, and the 4-page report map. No marketing / Fiverr / pricing / LinkedIn files in the repo.

Phase 12 — docs bundle

`docs/build_docs_bundle.py` (markdown → HTML → headless-Chrome PDF, green identity) → `docs/SwiftBite-Delivery-Documentation.pdf` (full 00–09 + walkthrough, 2.5 MB) and `docs/SwiftBite-Delivery-Summary.pdf` (scope + context + model + DQ + report structure, 0.9 MB). `docs/09_report_structure.md` was written here to complete the 00–09 set.

Project complete. Remaining polish: a Power BI Desktop spacing pass on the report (open `powerbi/SwiftBiteDelivery.pbip`, set `pDataFolder`, apply `theme.json`), and a Drive sync of the built PDFs.